Optimistic Asynchronous Dynamic-committee Proactive Secret Sharing

Bin Hu[1,*] Jianwei Liu[1,*] Zhenliang Lu[2,*] Qiang Tang[3,*] Zhuolun Xiang[4,*] Zongyang Zhang[1,*]

[1] Beihang University
{hubin0205, liujianwei, zongyangzhang}@buaa.edu.cn
[2] Nanyang Technological University
zhenliang.lu@ntu.edu.sg
[3] The University of Sydney
qiang.tang@sydney.edu.au
[4] Aptos Labs
xiangzhuolun@gmail.com

Abstract

Dynamic-committee Proactive Secret Sharing (DPSS) has gained increased attention for its ability to dynamically update shareholder committees and refresh secret shares, even under adversaries that gradually corrupt all nodes. However, existing state-of-the-art asynchronous DPSS protocols suffer from significant $O(n^3)$ message complexity and $O(\lambda n^3)$ communication complexity, where λ denotes the security parameter and n is the committee size.

In this paper, under the trusted setup assumption, we propose the first asynchronous DPSS protocol with $O(n^2)$ message complexity in all scenarios. Additionally, our protocol achieves $O(\lambda n^2)$ communication complexity in the optimistic case, where all nodes are honest and the network is synchronous, and $O(\lambda n^3)$ communication complexity in the worst case. Without a trusted setup, in the optimistic case, the message complexity is $O(n^2)$, and the communication complexity is $O(\lambda n^2 \log n)$. In the worst case, our protocol preserves state-of-the-art message and communication complexities, i.e., $O(n^3)$ and $O(\lambda n^3)$, respectively. Besides, our protocol supports batch amortization and accommodates high thresholds. For committee sizes of 4 to 400, the estimated concrete communication cost of our DPSS is 19–100x (resp., 8–14x) smaller in the optimistic case (resp., worst case) compared to the state-of-the-art. Experiments in AWS show that our DPSS achieves a latency of 1.9–8 seconds for committee sizes from 4 to 64. Single-machine benchmarks reveal a (computational) runtime reduction of up to 44%.

1 Introduction

Secret sharing [12, 56] is a well-established cryptographic primitive that has been extensively researched. The concept of secret sharing involves a dealer who possesses a secret s. The dealer's objective is to distribute secret shares among multiple nodes in such a way that a threshold number of shares is sufficient to reconstruct the original secret, whereas any subset of nodes less than the threshold gains no information about the secret. Secret sharing has found extensive applications in various domains, including Asynchronous Distributed Key Generation (ADKG) [24, 27, 43], Multi-Party Computation (MPC) [9, 20, 21, 47], Byzantine Fault Tolerant (BFT) protocols [37, 38, 60], and many others.

However, in the context of long-running applications, standard secret sharing faces the following challenges: (1) There may exist a strengthened adversary that is able to gradually compromise nodes, thus eventually accumulating sufficient shares to reconstruct the secret; (2) A long-lived system may encounter requests for membership alternation.

Recent works [40, 44, 59, 63] have been trying to tackle the above issues with asynchronous Dynamic-committee Proactive Secret Sharing (DPSS), where time is divided into multiple epochs, with shareholder committees potentially varying across the epochs, and secret shares being refreshed periodically. Since many state-of-the-art BFT protocols [32, 37, 38, 60] rely on threshold cryptosystems, asynchronous DPSS may gain its advantage in long-running BFT-based blockchain systems due to its robustness and ability to reconfigure shareholder committees, even in the presence of a strengthened adversary, node churn, and network fluctuation.

Although significant progress has been made on asynchronous DPSS, the recent state-of-the-art works "Long Live The Honey Badger" [63] (referred to as LongLive hereafter) and DyCAPS [40] still incur $O(\lambda n^3)$ communication complexity and $O(n^3)$ message complexity, where λ denotes the security parameter and n is the committee size. Here, communication complexity is the expected total number of bits transmitted by honest nodes, while message complexity refers

*Authors are listed alphabetically, Bin Hu & Zhenliang Lu led the effort with equal contributions. Zongyang Zhang is the corresponding author.

to the expected total number of messages exchanged among honest nodes.

We observe that LongLive and DyCAPS are suboptimal in terms of message complexity: Rambaud and Uban's work [44] (referred to as Y-VSS) achieves $O(n^2)$ message complexity, despite its higher communication complexity. Besides, almost all existing protocols (except Shanrang [59], see below) do not differentiate between optimistic and worst cases. That is, when all nodes are honest and the network is synchronous, such that all necessary messages are received before executing the next step, the performance remains the same as in the worst case, where Byzantine faults exist. Regarding Shanrang, despite its introduction of a two-path strategy, it experiences suboptimal fault tolerance of $n/4$ and exhibits $O(\lambda n^3 \log n)$ optimistic-case communication complexity. Hence, there is redundancy in the optimistic case for all existing asynchronous DPSS protocols. In addition, both LongLive and DyCAPS rely on a trusted setup to initialize the underlying cryptosystems for each epoch. Although LongLive proposes an approach to circumvent this trusted setup by employing an ADKG protocol, it makes DPSS have at least the same complexity as ADKG, which has a cost of $O(\lambda n^3)$ in communication complexity and $O(n^3)$ in message complexity. The above observations naturally lead to the following question:

Can we decrease the message complexity of asynchronous DPSS, while also reducing the communication complexity in the optimistic case? Besides, can we weaken or eliminate the need of the trusted setup for each epoch?

Our contributions. We address the aforementioned question by introducing an optimistic asynchronous DPSS protocol. Here is a summary of our contributions:

- We reduce the message complexity of asynchronous DPSS from $O(n^3)$ to $O(n^2)$ in both optimistic and worst cases, with a trusted setup assumption. Without a trusted setup, the optimistic-case message complexity remains $O(n^2)$, while the worst-case message complexity increases to $O(n^3)$, which still aligns with the state-of-the-art [63].
- With a trusted setup, we reduce the communication complexity to $O(\lambda n^2)$ in the optimistic case, whereas the worst-case communication complexity remains $O(\lambda n^3)$. Meanwhile, we also develop two variants to support batch amortization and high thresholds, respectively. For a committee size range of 4 to 400, the estimated concrete optimistic-case (resp., worst-case) communication cost is about 19–100x (resp., 8–14x) smaller than the state-of-the-art [63].
- We present two strategies to eliminate the strong assumption of per-epoch trusted setup. The first approach achieves a one-time trusted setup with a read-only bulletin board. The second approach eliminates the need for any trusted setup. The two approaches achieve communication complexities of $O(\lambda n^2)$ and $O(\lambda n^2 \log n)$ in the optimistic case, respectively, and their worst-case communication complexity is $O(\lambda n^3)$. The estimated concrete communication cost of our no-trusted-setup DPSS outperforms the state-of-the-art [63] (with per-epoch trusted setup) as early as $n > 13$ in the optimistic scenario.
- We implemented our DPSS protocol and deployed it on Amazon EC2 t2.medium instances across the globe.[1] Experimental results show that when the committee size n grows from 4 to 127, the transfer of shares between two committees of equal size takes around 1.9–20 seconds. Especially, for $n \leq 64$, our DPSS completes in less than 8 seconds. In single-machine benchmarks, our protocol saves up to 44% computation time when compared with the state-of-the-art [63], given $n \leq 52$.

The above improvements in message and optimistic-case communication complexities do not compromise the other asymptotic performance matrices of asynchronous DPSS.

2 Technical Overview

An asynchronous DPSS protocol comprises three sub-protocols: sharing, handoff, and reconstruction, referred to as DPSS.Share, DPSS.Handoff, and DPSS.Recon, respectively. Time is divided into epochs. In the initial epoch, DPSS.Share is executed, wherein a dealer distributes shares of a secret s among the nodes in the first committee $\mathcal{U}^1$. The DPSS.Recon sub-protocol is responsible for reconstructing the secret s at any epoch. The DPSS.Handoff sub-protocol is the key component of DPSS. Its primary objective is to ensure that a new committee obtains refreshed shares from an old committee while maintaining the confidentiality of the secret.

To simplify the expression, in epoch $e > 1$, we refer to the nodes in the old committee $\mathcal{U}^{e-1}$ and new committee $\mathcal{U}^e$ as "old nodes" and "new nodes", respectively. We use n_e to represent the size of $\mathcal{U}^e$ and t_e is the maximum number of faulty nodes that $\mathcal{U}^e$ can tolerate.[2] Given $n_e = 3t_e + 1$, a typical DPSS.Handoff protocol in epoch e consists of the following three phases:

- *Reshare:* Each old node reshares its old share with the new committee, ensuring that every new node outputs a valid sub-share of the old share.
- *Select:* The new committee collectively selects at least $t_{e-1} + 1$ valid resharing instances.
- *Compute:* Each new node computes its new share using the sub-shares obtained from the selected resharing instances.

[1] Available at https://anonymous.4open.science/r/DPSS_Impl.
[2] We consider $O(n_e) = O(t_e) = O(n)$ for any $e \in \mathbb{N}^*$.

Table 1: Performance metrics of asynchronous DPSS, where "—" means batched amortization is not supported.

Protocol	Tolerance	Dynamic	Communication Complexity			Message Complexity		Time Complexity⋆	High Threshold	Trusted Setup¶
			Optimistic†	Worst‡	Amortized	Optimistic†	Worst‡			
Cachin et al. [16]	$n/3$	×	$O(\lambda n^4)$	$O(\lambda n^4)$	—	$O(n^3)$	$O(n^3)$	$O(1)$	×	✓
Zhou et al. [66]	$n/3$	✓	$O(\exp(\lambda n))$	$O(\exp(\lambda n))$	—	$O(\exp(n))$	$O(\exp(n))$	$O(1)$	×	×
Shanrang [59]	$n/4$	✓	$O(\lambda n^3 \log n)$	$O(\lambda n^4)$	—	$O(n^3)$	$O(n^3)$	$O(\log n)$	✓	✓
Y-VSS [44]	$n/2$	✓	$O(\lambda n^4)$	$O(\lambda n^4)$	$O(\lambda n^3)$	$O(n^2)$	$O(n^2)$	$O(1)$	×	×
DyCAPS [40]	$n/3$	✓	$O(\lambda n^3)$	$O(\lambda n^3)$	—	$O(n^3)$	$O(n^3)$	$O(1)$	✓	✓
LongLive (hbACSS) [63]	$n/3$	✓	$O(\lambda n^3)$	$O(\lambda n^3)$	$O(\lambda n)$	$O(n^3)$	$O(n^3)$	$O(1)$	×	✓
Our DPSS (§ 7)	$n/3$	✓	$O(\lambda n^2)$	$O(\lambda n^3)$	$O(\lambda n)$ (optimistic)	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓
LongLive (DC-ACSS) [63]	$n/3$	✓	$O(\lambda n^3)$	$O(\lambda n^3)$	$O(\lambda n^3)$	$O(n^3)$	$O(n^3)$	$O(1)$	✓	×
LongLive$^{\nabla}$ (hbACSS) [63]	$n/3$	✓	$O(\lambda n^3)$	$O(\lambda n^3)$	$O(\lambda n^3)$	$O(n^3)$	$O(n^3)$	$O(\log n)$	×	×
Our DPSS (§ 9)	$n/3$	✓	$O(\lambda n^2 \log n)$	$O(\lambda n^3)$	$O(\lambda n \log n)$ (optimistic)	$O(n^2)$	$O(n^3)$	$O(1)$	✓	×

† Optimistic case: all nodes are honest and the network is synchronous.
‡ Worst case: up to $n/3$ nodes are Byzantine faulty and the network is asynchronous.
⋆ Time complexity: the expected number of asynchronous communication rounds of the protocol.
¶ LongLive demonstrates that employing ADKG protocols can eliminate the need for trusted setups for threshold signatures, without affecting the asymptotic performance of its DC-ACSS-based version. However, the hbACSS-based LongLive still requires trusted setups to generate the powers of tau (t-SDH parameters) for KZG commitments.
$^{\nabla}$ In LongLive$^{\nabla}$ (hbACSS), we eliminate all trusted setups by employing an ADKG protocol for threshold signatures and leveraging the work of Das et al. [26] to generate the powers of tau parameters for KZG commitments, incurring a time complexity of $O(\log n)$.

Existing asynchronous DPSS.Handoff protocols, such as those proposed by Hu et al. [40] and Yurek et al. [63], employ $O(n)$ parallel Asynchronous Complete Secret Sharing (ACSS) instances for the *Reshare* phase and a single Multi-valued Validated Byzantine Agreement (MVBA) instance for the *Select* phase. The selection is based on the condition that, from each selected ACSS instance, at least one new honest node has received a valid share. The completeness property of ACSS guarantees that if one honest node outputs a valid share, all honest nodes will output valid shares. Therefore, all honest new nodes eventually obtain valid shares from all the selected ACSS instances. Finally, in the *Compute* phase, the honest new nodes locally compute the refreshed shares using the outputs of selected ACSS instances.

Bottlenecks. We observe that ACSS, while being a robust component in DPSS.Handoff, is a dominant bottleneck in message and communication complexities. Specifically, many ACSS protocols [6, 7, 62] use exactly the same communication pattern as Bracha's reliable broadcast [14], consuming at least $O(n^2)$ messages due to the Dolev-Reischuk bound [29]. Hence, $O(n)$ parallel ACSS instances incur $O(n^3)$ message complexity and $O(\lambda n^3)$ bits of communication. These costs align with the performance of the state-of-the-art [40, 63]. Following the identified bottlenecks, our approach to designing a more efficient asynchronous handoff protocol is as follows.

Reducing the message complexity by provable sharing and batched recovery. We replace ACSS with a Weak Provable ACSS (wpACSS) protocol, consisting of a provable sharing sub-protocol and an optional recovery sub-protocol.

The provable sharing is inspired by provable broadcast [2, 37], where the dealer generates a proof of sharing that guarantees the receipt of valid shares by at least $n-2t$ honest nodes ($n = 3t+1$). As illustrated in Figure 1, the dealer shares its secret via SHARE messages. Upon receiving a valid secret share from the dealer, each node responds with an acknowledgment of receipt. The dealer aggregates the acknowledgments from $2t+1$ nodes and forms an intermediate proof, which demonstrates that at least $t+1$ honest nodes have received valid shares. This proof is then multicast to all nodes. When a node receives this proof, it accepts the contents of the SHARE message as its output. Meanwhile, it sends back a confirmation to the dealer, indicating that it has output in the provable sharing. Finally, the dealer aggregates confirmations from $2t+1$ nodes and forms a proof of sharing. This proof ensures that at least $t+1$ honest nodes have output. This process maintains an efficient $O(n)$ message complexity.

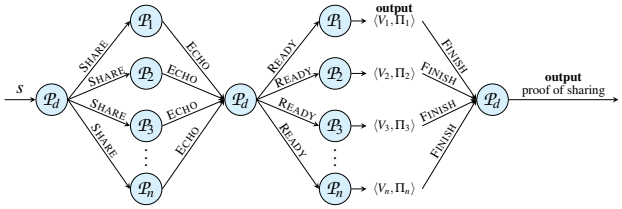

Figure 1: Message flow of provable sharing wpACSS.Share.

The optional recovery sub-protocol enables a node to recover its missing share, as long as a valid proof of sharing exists. When a node requests share recovery, at least $t+1$ honest nodes respond with HELP messages, leading to an $O(n)$ message complexity. For each sharing instance, there are at most t honest nodes missing their deserved shares. As

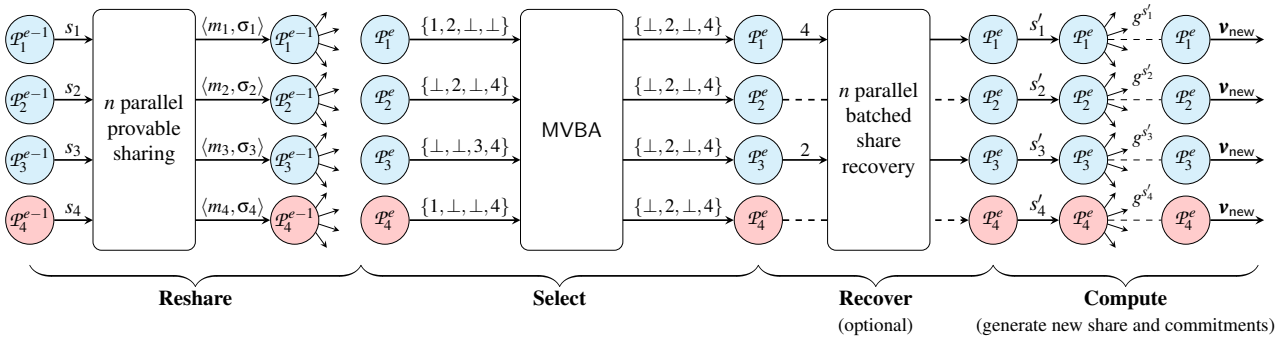

Figure 2: Overview of our DPSS.Handoff between two committees with 4 nodes, where nodes $\mathcal{P}_4^{e-1}$ and $\mathcal{P}_4^{e}$ are corrupted.

we will employ n parallel provable sharing instances in our handoff protocol, if we do the recovery naively, there will be at most $O(n^2)$ recovery instances invoked, resulting in a cubic message complexity. To obtain a handoff protocol with $O(n^2)$ message complexity, the key observation is that we can batch the recovery requests and responses. Namely, we let each node aggregate the indexes of missing shares into one request, with each index referring to a dealer. Similarly, every honest node responds with a single message to help the recovery. For each requested index, there are at least $t+1$ responses that can help to recover the missing share. Hence, the $O(n^2)$ recovery instances are replaced with $O(n)$ batched recovery instances, each incurring $O(n)$ messages. In this way, even if $O(n)$ batched recovery instances are invoked in our handoff protocol, the message complexity is still $O(n^2)$.

As shown in Figure 2, our DPSS.Handoff protocol comprises multiple components: n parallel instances of provable sharing, an instantiation of MVBA, n parallel batched recovery instances (optional), and the computation of new shares and commitments. By our design, every provable sharing and batched recovery instance has $O(n)$ message complexity. Besides, we employ the MVBA protocol by Lu et al. [48], which has an $O(n^2)$ message complexity. The final computation component also incurs $O(n^2)$ message complexity. Hence, we reduce the overall message complexity to $O(n^2)$.

Reducing optimistic-case communication complexity by minimizing message lengths. Although we have made the share recovery sub-protocol optional, it does not help reduce the communication complexity of handoff in the optimistic case: if a SHARE message in provable sharing has a length of $O(\lambda n)$ bits, n provable sharing instances still cost $O(\lambda n^3)$ communication complexity.

It is noteworthy that the SHARE messages in most existing ACSS protocols are typically of linear size. For example, in AVZ21 [6], a SHARE message contains one secret share and n recovery elements. The recovery elements serve as masks during the recovery, ensuring that each node can only recover its own share and learns no knowledge of the other shares. The ACSS protocol by Das et al. [27] and its variant in LongLive [63] also have $O(\lambda n)$-sized SHARE messages, as they gather the encryptions of n shares into one message. Hence, a naive provable sharing protocol derived from these ACSS protocols will also have linear-sized SHARE messages.

To improve the optimistic-case communication complexity, we propose a new design of provable sharing with $O(\lambda)$-sized SHARE messages. We are inspired by the Verifiable Secret Sharing with Recovery (VSSR) protocol proposed by Basu et al. [8]. The main idea is to interpolate Distributed Pseudo-random Function (DPRF) evaluations into $\ell = \lceil n/t \rceil$ recovery polynomials, instead of sampling n recovery polynomials as in [6]. The evaluations of recovery polynomials serve as the masks for the share recovery. Now, a SHARE message contains one secret share and ℓ recovery elements, where ℓ is a constant. Furthermore, every recovery element is accompanied by a constant-sized verifiable proof. As a result, we reduce the size of SHARE messages from $O(\lambda n)$ to $O(\lambda)$.

However, applying the VSSR protocol in our provable sharing is not straightforward. In the original VSSR protocol [8], the distribution of public information of DPRF among the nodes is not explicitly specified. If we let the dealer directly send the $O(\lambda n)$-sized DPRF public keys $\{dpk_i\}_{i\in[n]}$ (used to verify DPRF contributions) to all nodes, it would result in a communication complexity of at least $O(\lambda n^2)$ for each provable sharing. To address this issue, we carefully design our provable sharing protocol to incorporate the DPRF keys within the SHARE messages, keeping the message length constant. In this way, we achieve a provable sharing protocol with $O(\lambda n)$ communication complexity.

Now that we have an efficient provable sharing protocol, the next task is to avoid old nodes resharing incorrect shares. We let the new node use a commitment vector to validate the proofs of sharing. The vector consists of the commitments to all old shares. At the end of handoff, the new committee jointly generates a new commitment vector of the refreshed shares. However, the generation of such a vector is non-trivial, as we have to maintain an $O(\lambda n^2)$ optimistic-case communication complexity. We propose a two-path strategy for this purpose. Specifically, the new nodes first try to communicate the new commitments in an optimistic way, which consumes $O(\lambda n^2)$ bits of communication. If any misbehavior is detected, they fallback into a pessimistic but robust path, at the cost of $O(\lambda n^3)$ communication complexity.

3 Related Work

In a long-running system based on secret sharing, there may exist a strengthened adversary that gradually corrupts the nodes. Such an adversary could eventually accumulate the necessary number of shares to reconstruct the secret, thus compromising confidentiality. To address this risk, Herzberg et al. [39] introduce the concept of Proactive Secret Sharing (PSS), which involves periodic share refreshes to constrain the power of such an adversary. That is, to obtain the secret value, the adversary has to corrupt more than a threshold of nodes within a limited time. Desmedt and Jajodia [28] extend PSS to the dynamic committee setting, where nodes are allowed to join and leave the system dynamically. They introduce the concept of Dynamic-committee Proactive Secret Sharing (DPSS), aimed to adapt to changes in committee composition and ensure the continuous security of long-running systems.

Non-asynchronous DPSS. CHURP [49] is a synchronous protocol that achieves a communication complexity of $O(\lambda n^2)$ bits and a message complexity of $O(n^2)$ in the all-honest case. However, when faced with crash or Byzantine faults, its communication and message complexity increase to $O(\lambda n^3)$ and $O(n^3)$, respectively. Goyal et al. [36] propose a synchronous protocol that enhances CHURP by a factor of $O(n)$ in the batched setting. In the single-secret setting, their protocol has the same communication cost as CHURP. COBRA [58] is a state-of-the-art partially synchronous DPSS protocol that achieves an improvement over Schultz-MPSS [55]. Schultz-MPSS has a communication complexity of $O(\lambda n^4)$ and a message complexity of $O(n^3)$ across all cases. In contrast, COBRA achieves a communication complexity of $O(\lambda n^3)$ and a message complexity of $O(n^2)$ in the optimistic case.

Asynchronous DPSS. The asynchronous setting has garnered significant interest in recent years. Table 1 shows the performance metrics of related asynchronous DPSS protocols. The earliest related work on asynchronous PSS is proposed by Cachin et al. [16], which only supports the static committee setting. Zhou et al. [66] introduce the first DPSS protocol, but it has an exponential communication cost. Recently, Shanrang [59] has made remarkable improvements to the asynchronous DPSS protocol. In the case of all honest participants, its communication complexity is reduced to $O(\lambda n^3 \log n)$, and when facing adversaries, its communication complexity grows to $O(\lambda n^4)$. However, this improvement comes at the expense of suboptimal resilience, i.e., $t < n/4$. A recent work Y-VSS [44] tolerates up to $1/2$ corrupted nodes in an asynchronous network. However, it assumes one of the following conditions: (1) the dealer is honest, (2) the initial network functions synchronously, or (3) the first committee has at most $1/3$ corrupted nodes. Additionally, it requires non-overlapping old and new committees, which imposes additional conditions compared to the assumptions in our paper.

The current state-of-the-art asynchronous DPSS protocols are LongLive [63] and DyCAPS [40], both achieving $O(\lambda n^3)$ communication complexity and $O(n^3)$ message complexity. LongLive follows the reshare-then-agree approach by Cachin et al. [16], and it simultaneously supports high-threshold and batched DPSS. On the other hand, DyCAPS utilizes bivariate polynomials to achieve high-threshold DPSS. However, DyCAPS loses the support for batched DPSS, even though it has a lower concrete communication overhead in the non-batched setting. As mentioned in Section 1, both LongLive and DyCAPS do not distinguish the performance between optimistic and pessimistic cases, and their message complexity is suboptimal when compared with Y-VSS [44].

4 Model and Design Goal

4.1 System Model

Established identities. The i-th node within committee $\mathcal{U}^e$ is assigned a distinct identity $\mathcal{P}_i^e$. An honest node is considered active in epoch e if it is a member of committee $\mathcal{U}^e$. If a node is active in two different epochs, it will have two distinct identities. The relationship between two committees can range from overlap to complete disjointedness.

Trusted setup. For each epoch, a trusted setup is required for initializing the cryptosystems.[3] This entails generating threshold key shares for the new committee, which are used to generate threshold signatures. The trusted setup is also responsible for generating the public parameters for the commitment schemes, e.g., powers of tau [26] for KZG commitments. We assume that the public information is available to all nodes, and every node in epoch e' has access to all public information pertaining to all previous epochs $e < e'$.

Adversary model. We assume a static probabilistic polynomial time (PPT) adversary. It can corrupt up to $t_e < n_e/3$ nodes in each committee $\mathcal{U}^e$ at the beginning of epoch e, where n_e is the number of nodes in $\mathcal{U}^e$. A corrupted node is Byzantine faulty [1], which means it may misbehave arbitrarily. Throughout this paper, we assume $n_e = 3t_e + 1$.

If a node is honest in epoch $e-1$ ($e > 1$) and gets corrupted in epoch e, then the adversary can only access its private information assigned for epoch e, and the private information within epoch $e-1$ remains unknown (e.g., encrypted). Nonetheless, the adversary can delete previous information held by the corrupted node. Consequently, a corrupted node may lose its old share and thus fail to participate in the handoff protocol. Hence, if there is an overlap between committees $\mathcal{U}^{e-1}$ and $\mathcal{U}^e$, we assume that at least $t_{e-1}+1$ nodes in $\mathcal{U}^{e-1}$ still have their old shares, i.e., not corrupted in both epochs $e-1$ and e.

We adopt the approach outlined in Schultz-MPSS [55] for defining epochs and imposing constraints on the adversary, which helps overcome the impossibility result presented

[3] We discuss in Section 9 that achieving a one-time trusted setup or transparent setup is feasible in our protocol.

by Alexandru et al. [4]. For a more comprehensive understanding of the model, additional information is available in Appendix A of the paper [63].

Network model. We assume authenticated and point-to-point channels, ensuring that recipients can verify senders' identities. The adversary can control the order of messages but cannot observe or tamper with the contents.

Additionally, we consider that our protocol operates under the same network conditions as described in [57], encompassing the optimistic case (referred to as the common case in [57]) and the worst case. For epoch e, the optimistic case signifies that all nodes are honest and the network is synchronous, while the worst case denotes the presence of up to t_e corrupted nodes and the network is asynchronous. Here, synchrony means that messages among honest nodes are delivered within a known time interval Δ, whereas asynchrony implies that all messages will be delivered without any time constraints. Furthermore, in both cases, once a node completes the handoff, it waits for a specified period (measured by its local clock) before joining the next epoch as part of the old committee. It is important to note that nodes may begin the next epoch at different times if their completion times differ. When we refer to the optimistic case, we explicitly assume all nodes have received necessary messages before proceeding to the next step, and we omit the parameter Δ.

4.2 Design Goal

Our goal is to design an efficient asynchronous DPSS protocol, which consists of three sub-protocols: sharing, handoff, and reconstruction. The sharing sub-protocol DPSS.Share is only invoked in epoch $e = 1$, whereas the reconstruction sub-protocol DPSS.Recon can be invoked at any time after sharing. Between epochs $e-1$ and e ($e > 1$), the handoff sub-protocol DPSS.Handoff is executed to facilitate the transfer of shares from the old committee to the new committee. Our main emphasis is on the efficiency of the handoff protocol, which is executed periodically and has a significant message and communication cost. The details of DPSS.Share and DPSS.Recon are deferred to Appendix B.1.

Formally, an asynchronous DPSS.Handoff protocol with epoch e satisfies the following properties with all but negligible probability.

- *Termination.* If all honest nodes in committees $\mathcal{U}^{e-1}$ and $\mathcal{U}^e$ invoke DPSS.Handoff, then all honest nodes in $\mathcal{U}^{e-1}$ eventually halt, and all honest nodes in $\mathcal{U}^e$ eventually have outputs.
- *Correctness.* The secret s stays invariant across the executions of DPSS.Handoff.
- *Secrecy.* Given a uniformly random secret s, a static PPT adversary does not gain any additional knowledge about the secret from DPSS.Handoff beyond g^s.

In this paper, we make the assumption that all honest nodes within the first committee $\mathcal{U}^1$ have already produced valid shares and commitments from DPSS.Share. We sometimes use the handoff sub-protocol DPSS.Handoff to represent the overall DPSS protocol when the context is unambiguous.

Throughout this paper, we assume that the secret s is uniformly random. It is worth noting that many typical applications of DPSS are not constrained by this uniformly random requirement, including: (1) Threshold cryptosystems, where secret keys are inherently uniformly random. (2) The "BMR Escape Hatch" described in [63], which relies on uniformly random values as secrets. (3) Cloud storage of confidential files, such as in COBRA [58], where files can be encrypted, and the uniformly random encryption keys can be protected using DPSS. We will give more details on how to protect a non-uniform secret using DPSS in Section 9.

5 Notations and Preliminaries

5.1 Notations

Let $\mathbb{G}$ denote a multiplicative cyclic group of prime order q, where the discrete logarithm problem is assumed to be hard, and g is a generator of $\mathbb{G}$. The secret value is referred to as $s \in \mathbb{Z}_q$. For an integer $a \in \mathbb{N}^*$, $[a]$ denotes the ordered set $\{1, 2, \cdots, a\}$. Throughout this paper, we use $|\mathcal{S}|$ to capture the size of a set $\mathcal{S}$. The cryptographic security parameter is denoted as λ, which captures the length of a (threshold) signature or a hash value.The epoch number is denoted as e, where $e \in \mathbb{N}^*$. The committee in the e-th epoch is represented as $\mathcal{U}^e = \{\mathcal{P}_i^e\}_{i\in[n_e]}$, where $\mathcal{P}_i^e$ is the i-th node and $n_e = |\mathcal{U}^e|$. When the context is unambiguous, we sometimes use $\mathcal{P}_i$ and $\mathcal{P}_i'$ to refer to nodes $\mathcal{P}_i^{e-1}$ and $\mathcal{P}_i^e$, respectively. Similarly, we will use $\mathcal{U}$ to represent the old committee and $\mathcal{U}'$ to represent the new committee. Besides, t_e is the maximum number of corrupted nodes in epoch e. The threshold full signature of a message m is denoted as σ_m, and $\sigma_{m,i}^*$ is a signature share (also called partial signature) produced by $\mathcal{P}_i$. We use PC (resp., VC) to denote a polynomial (resp., vector) commitment, and w (resp., π) refers to a witness of a polynomial evaluation (resp., opening of a vector element). A comprehensive list of frequently used symbols is provided in Table 2.

5.2 Preliminaries

Our design employs several cryptographic primitives. Here, we only provide the abstractions of these protocols, and the formal definitions can be found in Appendix A.

Polynomial commitment (PCom) [33, 41] allows a node to commit to a polynomial and give witnesses of its evaluations. It consists of four algorithms: PCom.Setup, PCom.Commit, PCom.Witness, and PCom.VerifyEval. PCom.Setup generates public parameters pp, which is sometimes omitted for simplicity. PCom.Commit generates a commitment PC_ϕ for

Table 2: Notations used in this paper

Notation	Description
λ	Security parameter
q	Prime
$\mathbb{G}$	Multiplicative cyclic group of prime order q
g	Generator of group $\mathbb{G}$
s	Secret value
g^s	Commitment to secret s
e	Epoch number
$\mathcal{U}^e$	Shareholder committee in epoch e (sometimes simplified as $\mathcal{U}$ or $\mathcal{U}'$)
$\mathcal{P}_i^e$	i-th node in $\mathcal{U}^e$, sometimes simplified as $\mathcal{P}_i$ or $\mathcal{P}_i'$
s_i, s_i'	Secret shares of $\mathcal{P}_i$ and $\mathcal{P}_i'$
n_e	Size of committee $\mathcal{U}^e$, sometimes simplified as n or n'
t_e	Threshold in epoch e, sometimes simplified as t or t'
$\boldsymbol{\nu}$	Vector
$VC_{\boldsymbol{\nu}}$	Vector commitment to a vector $\boldsymbol{\nu}$
$\pi^i_{\boldsymbol{\nu}}$	Opening proof of the i-th element in vector $\boldsymbol{\nu}$
PC_ϕ	Polynomial commitment to a polynomial ϕ
w^i	Witness of the i-th evaluation of polynomial ϕ
$\mathcal{H}$	Hash function
ID	Session identifier
$\Pi[\mathsf{ID}](x)$	Instance of Π identified by ID with input x
tsk, tpk	Secret and public keys in threshold signature (TS)
tsk_i, tpk_i	Node $\mathcal{P}_i$'s secret and public key shares in TS
$\sigma^*_{m,i}$	Signature share on message m produced by $\mathcal{P}_i$
σ_m	Threshold full signature of message m
dsk, dpk	Secret and public keys in distributed pseudorandom function (DPRF)
dsk_i, dpk_i	Node $\mathcal{P}_i$'s secret and public key shares in DPRF
$\boldsymbol{\nu}_{\mathsf{old}}, \boldsymbol{\nu}_{\mathsf{new}}$	Commitment vectors to the old and new shares

the polynomial $\phi(x) \in \mathbb{Z}_q[x]$. For $x = i$, PCom.Witness outputs an evaluation $\phi(i)$ and a witness w^i. PCom.VerifyEval is then used to verify the correctness of the evaluation. In this paper, we utilize the KZG scheme [41] for PCom because of its constant commitment and witness sizes.

Verifiable secret sharing (VSS) [6, 52] enables a dealer to distribute a secret among n nodes, allowing each node to independently verify the correctness of its share. It involves four algorithms: VSS.Share, VSS.Vrfy, VSS.Recon, and VSS.MakeSecret. Given a threshold t, a committee size n, a secret s, and a security parameter λ, VSS.Share produces a polynomial commitment PC_s and n share-witness pairs $\langle s_i, w_s^i \rangle_{i\in[n]}$. VSS.Vrfy verifies the shares. VSS.Recon reconstructs the secret from a sufficient number of shares. Lastly, taking as inputs a threshold t, an index set I, and a set of values $\{y_i\}_{i\in I}$, where $|I| \le t$, VSS.MakeSecret produces a degree-t polynomial $\phi(x)$ by locally sampling $t+1-|I|$ points, such that $\phi(i) = y_i$ for every $i \in I$. The above algorithms are all non-interactive, making them oblivious to the network model.

Vector commitment (VCom) [19, 35, 45] allows a node to commit to a vector and later reveal the vector elements for a specific set of positions. It comprises four algorithms: VCom.Setup, VCom.Commit, VCom.Open, and VCom.Vrfy. Given a vector size n and a security parameter λ, VCom.Setup generates public parameters pp (sometimes omitted for simplicity). VCom.Commit produces a commitment $VC_{\boldsymbol{\nu}}$ to vector $\boldsymbol{\nu}$. VCom.Open yields the i-th element v_i in $\boldsymbol{\nu}$ and an opening proof $\pi^i_{\boldsymbol{\nu}}$. Taking as inputs a commitment $VC_{\boldsymbol{\nu}}$, an index i, an element v_i, and a proof $\pi^i_{\boldsymbol{\nu}}$, VCom.Vrfy outputs 1 iff v_i is the i-th element of $\boldsymbol{\nu}$. In this paper, we employ Pointproofs [35] for VCom, due to its constant commitment and proof sizes.

Threshold signature (TS) [13] enables a quorum of nodes to collaboratively sign a message. It includes five algorithms: TS.KeyGen, TS.SignShare, TS.VrfyShare, TS.Combine, and TS.Vrfy. Taking as inputs a threshold t and a committee size n, TS.KeyGen generates threshold signing key pairs $\{tpk, \langle tpk_i, tsk_i\rangle_{i\in[n]}\}$. $\mathsf{TS.SignShare}_t$ creates a signature share $\sigma^*_{m,i}$ for a message m using $\mathcal{P}_i$'s secret key share tsk_i, while $\mathsf{TS.VrfyShare}_t$ verifies a signature share by $\mathcal{P}_i$'s public key share tpk_i and the message m. $\mathsf{TS.Combine}_t$ generates a full signature from at least $t+1$ valid signature shares, and $\mathsf{TS.Vrfy}_t$ is employed to verify a full signature using tpk. We sometimes omit the key pairs for simplicity.

Distributed pseudo-random function (DPRF) [3, 51] enables a quorum of nodes to jointly evaluate a pseudo-random function. It has six algorithms: DPRF.Init, DPRF.Contrib, DPRF.VrfyContrib, DPRF.Combine, DPRF.Eval, and DPRF.Vrfy. Taking as inputs a threshold t, a committee size n, and a security parameter λ, DPRF.Init not only initializes the DPRF key pairs $\langle dsk, dpk, \{dsk_i, dpk_i\}_{i\in[n]}\rangle$, but it also generates a polynomial commitment PC_{dsk}, a vector commitment VC_{dpk}, and n tuples $\langle w^i_{dsk}, \pi^i_{dpk}\rangle_{i\in[n]}$ to facilitate the verifiable distribution of DPRF key pairs. Taking a value x and a DPRF key pair $\langle dsk_i, dpk_i\rangle$ as inputs, DPRF.Contrib yields a DPRF contribution $F_i(x)$ and a proof $\mathsf{Proof}^i_{\mathsf{dprf}}$. DPRF.VrfyContrib verifies whether a DPRF contribution is correctly generated. DPRF.Combine combines $t+1$ valid contributions into a DPRF evaluation $y = F(x)$, whereas DPRF.Eval directly outputs $y = F(x)$ when provided with a value x and a DPRF secret key dsk. DPRF.Vrfy verifies DPRF evaluations. We provide a DPRF instantiation in Algorithm 5, Appendix A, adapted from Figure 6 in [3].

Multi-valued validated Byzantine agreement (MVBA) protocol [2, 17, 48] enables a group of nodes to submit individual proposals and reach an agreement on an output that satisfies a global predicate P_{MVBA}. In this paper, we utilize the MVBA protocol by Lu et al. [48]. This protocol offers $O(n|m| + \lambda n^2)$ communication complexity, $O(n^2)$ message complexity, and $O(1)$ time complexity, where $|m|$ represents the input size.

We remark that the MVBA protocol in [48] relies on a common coin sub-protocol to generate common randomness. In Section 9, we will introduce a new MVBA protocol that incorporates an additional *good-case-no-coin* property. This property ensures that no randomness is required in the optimistic case, which can effectively eliminate the need for common coins in such scenarios without impacting the performance metrics. We refer to this enhanced MVBA protocol

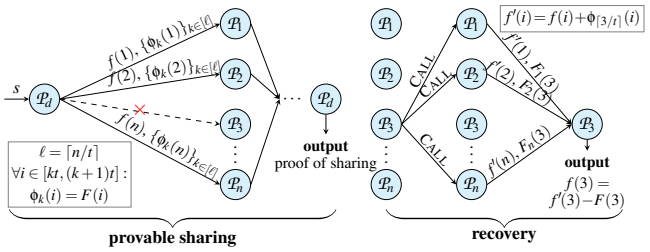

Figure 3: Overview of our wpACSS protocol.

as *good-case-no-coin* MVBA.

6 Weak Provable ACSS (wpACSS)

As mentioned earlier, we aim to avoid directly invoking ACSS in DPSS.Handoff to mitigate the high message complexity. Additionally, we desire to achieve a constant message size during the resharing phase. Furthermore, every honest node should be able to recover its missing share without significantly increasing the overall communication complexity. In this section, we introduce our Weak Provable ACSS (wpACSS) protocol, which consists of two sub-protocols:

- **Provable sharing:** it allows a dealer $\mathcal{P}_d$ to share a uniformly random secret s with n nodes. If the dealer outputs a proof of sharing, then at least $t+1$ honest nodes have output valid shares.
- **Recovery:** it allows a node to recover its share with the help of other nodes. If there exists a valid proof of sharing, an honest node is guaranteed to output a valid share through recovery.

Formally, a wpACSS protocol satisfies the following properties except with negligible probability.

- *Termination.* If the dealer $\mathcal{P}_d$ is honest, then all honest nodes eventually output valid shares in the provable sharing sub-protocol, and $\mathcal{P}_d$ outputs a proof of sharing.
- *Provability.* If the dealer $\mathcal{P}_d$ outputs a valid proof of sharing, then at least $t+1$ honest nodes output valid shares in the provable sharing sub-protocol. These honest nodes are able to help others to recover shares in the recovery sub-protocol, so that all honest nodes eventually output valid shares.
- *Correctness.* If a dealer $\mathcal{P}_d$ inputs a secret s in the provable sharing sub-protocol and outputs a valid proof of sharing, then there exists a fixed secret $\hat{s}$, such that any $t+1$ shares from honest nodes can reconstruct $\hat{s}$. If the dealer is honest, then $s=\hat{s}$.
- *Secrecy.* If the dealer $\mathcal{P}_d$ is honest and it shares a uniformly random secret s in wpACSS, then a computationally bounded adversary learns no extra information about the secret beyond g^s.

We remark that the key difference between our wpACSS and other ACSS protocols is that, our wpACSS exhibits a "weaker" completeness property. As mentioned in Section 2, the completeness property of ACSS ensures that if one honest node outputs a valid share, all honest nodes will output valid shares. In contrast, our wpACSS only guarantees that if the dealer produces a proof of sharing, at least $t+1$ honest nodes will output valid shares. The remaining honest nodes are guaranteed to recover their shares through the recovery sub-protocol.

6.1 Overview of our wpACSS Protocol

Our provable sharing sub-protocol wpACSS.Share has an $O(n)$ message complexity. To further achieve constant-sized SHARE messages, we draw inspiration from VSSR [8]. At a high level, a dealer $\mathcal{P}_d$ generates one sharing polynomial $f(x)$ and ℓ recovery polynomials $\{\phi_k(x)\}_{k\in[\ell]}$, where $\ell=\lceil n/t\rceil$ is a constant. The sharing polynomial encodes the secret into shares, while every recovery polynomial interpolates t random masks (i.e., DPRF evaluations) for potential share recovery. As shown in Figure 3, $\mathcal{P}_d$ sends to each node $\mathcal{P}_i$ a secret share $f(i)$ and ℓ evaluation points $\{\phi_k(i)\}_{k\in[\ell]}$ via a SHARE message. Each value is accompanied by a constant-sized proof. Additionally, $\mathcal{P}_d$ distributes the DPRF key shares $\langle dsk_i, dpk_i\rangle_{i\in[n]}$ among the nodes. The distribution procedure is carefully designed, without asymptotically increasing the length of SHARE messages. The proofs and DPRF key shares are omitted in Figure 3.

When an honest node $\mathcal{P}_i$ observes a valid proof of sharing but has not output a valid share from wpACSS.Share, it invokes the recovery sub-protocol. Every other honest node $\mathcal{P}_j$, where $j\in[n]$, responds with a masked share $f'(j)=f(j)+\phi_{\lceil i/t\rceil}(j)$ and a DPRF contribution $F_j(i)$. $\mathcal{P}_i$ interpolates the masked polynomial $f'(x)=f(x)+\phi_{\lceil i/t\rceil}(x)$ using $t+1$ masked shares and combines $t+1$ DPRF contributions into a DPRF evaluation $F(i)$. Since the recovery polynomial $\phi_{\lceil i/t\rceil}(x)$ is constructed subject to $\phi_{\lceil i/t\rceil}(i)=F(i)$, $\mathcal{P}_i$ can recover its share as $f(i)=f'(i)-F(i)$.

Moreover, to reduce the message complexity when $O(n)$ parallel share recovery instances are invoked, the messages can be batched. Specifically, a node may request share recovery for multiple shares within a single message, and every other node responds with the masked shares and DPRF evaluations in a single message as well.

6.2 Details of our wpACSS Protocol

Provable sharing. The details of wpACSS.Share are shown in Algorithm 1, which comprises four logical phases. In this sub-protocol, we refer to the dealer and committee as $\mathcal{P}_d$ and $\mathcal{U}$, respectively, and the session identifier is ID.

1. Share. In this first phase, the dealer $\mathcal{P}_d$ sends a SHARE message to each node $\mathcal{P}_i\in\mathcal{U}$ (lines 1–15). Specifically, $\mathcal{P}_d$ first generates secret shares $\{s_i\}_{i\in[n]}$ of the secret s using

Algorithm 1 wpACSS.Share$[\langle \mathsf{ID}, d\rangle]$, with $\mathcal{P}_d$ as the dealer

Let $\mathcal{H}: \mathbb{G} \to \mathbb{Z}_q$ be a hash function; $\ell \leftarrow \lceil n/t \rceil$

As a dealer $\mathcal{P}_d$:

1: **upon** receiving $(\mathcal{U}, t, n, s)$ as input **do**
2: $S_{\mathsf{sig},1} \leftarrow \emptyset$; $S_{\mathsf{sig},2} \leftarrow \emptyset$
3: $\{PC_s, \langle s_i, w_s^i\rangle_{i\in[n]}\} \leftarrow \mathsf{VSS.Share}(t,n,s)$
4: let $f(x)$ be the sharing polynomial, s.t. $f(0)=s$ and $f(i)=s_i, \forall i \in [n]$
5: $z(x) \leftarrow f(x) - s$
6: $PC_z \leftarrow \mathsf{PCom.Commit}(z(x))$
7: $\langle z(0), w_z^0\rangle \leftarrow \mathsf{PCom.Witness}(z(x), 0)$
8: $\langle dsk_i, \{\phi_k(i)\}_{k\in[\ell]}, \mathsf{Proof}^i_{\mathsf{rec}}\rangle_{i\in[n]} \leftarrow \mathsf{GenRecPoly}[\langle \mathsf{ID}, d\rangle](t,n)$
9: $\boldsymbol{v}_s \leftarrow \{g^{s_i}\}_{i\in[n]}$
10: $VC_s \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}_s)$
11: **for** each $\mathcal{P}_i \in \mathcal{U}$ **do**
12: $\langle g^{s_i}, \pi_s^i\rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}_s, i)$
13: $V_i \leftarrow \langle s_i, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle$
14: $\Pi_i \leftarrow \langle g^s, PC_s, w_s^i, PC_z, w_z^0, VC_s, \pi_s^i, \mathsf{Proof}^i_{\mathsf{rec}}\rangle$
15: send $\langle \textsc{Share}, \mathsf{ID}, V_i, \Pi_i\rangle$ to $\mathcal{P}_i$ ▷ share
16: **upon** receiving $\langle \textsc{Echo}, \mathsf{ID}, \sigma^*_{\mathsf{r},d,i}\rangle$ from $\mathcal{P}_i \in \mathcal{U}$ for the first time **do**
17: $m_d \leftarrow \mathsf{ID}||d||g^s||PC_s||VC_s||PC_{dsk}||VC_{dpk}||\{PC^k_\phi\}_{k\in[\ell]}$
18: **if** $\mathsf{TS.VrfyShare}_{n-t}(\textsc{ready}||m_d, i, \sigma^*_{\mathsf{r},d,i}, tpk_i) = 1$ **then**
19: $S_{\mathsf{sig},1} \leftarrow S_{\mathsf{sig},1} \cup \langle i, \sigma^*_{\mathsf{r},d,i}\rangle$
20: **if** $|S_{\mathsf{sig},1}| = n-t$ **then**
21: $\sigma_{\mathsf{r},d} \leftarrow \mathsf{TS.Combine}_{n-t}(\textsc{ready}||m_d, S_{\mathsf{sig},1})$ ▷ ready
22: multicast $\langle \textsc{Ready}, \mathsf{ID}, m_d, \sigma_{\mathsf{r},d}\rangle$
23: **upon** receiving $\langle \textsc{Finish}, \mathsf{ID}, \sigma^*_{\mathsf{f},d,i}\rangle$ from $\mathcal{P}_i \in \mathcal{U}$ for the first time **do**
24: **if** $\mathsf{TS.VrfyShare}_{n-t}(\textsc{finish}||m_d, i, \sigma^*_{\mathsf{f},d,i}, tpk_i) = 1$ **then**
25: $S_{\mathsf{sig},2} \leftarrow S_{\mathsf{sig},2} \cup \langle i, \sigma^*_{\mathsf{f},d,i}\rangle$
26: **if** $|S_{\mathsf{sig},2}| = n-t$ **then**
27: $\sigma_{\mathsf{f},d} \leftarrow \mathsf{TS.Combine}_{n-t}(\textsc{finish}||m_d, S_{\mathsf{sig},2})$ ▷ finish
28: **output** $\langle m_d, \sigma_{\mathsf{f},d}\rangle$ ▷ output a proof of sharing, only for dealer

As a node $\mathcal{P}_i \in \mathcal{U}$:

29: **upon** receiving $\langle \textsc{Share}, \mathsf{ID}, V_i, \Pi_i\rangle$ from $\mathcal{P}_d$ for the first time **do**
30: parse V_i as $\langle s_i, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle$
31: parse Π_i as $\langle g^s, PC_s, w_s^i, PC_z, w_z^0, VC_s, \pi_s^i, \mathsf{Proof}^i_{\mathsf{rec}}\rangle$
32: **if** $PC_s = PC_z g^s \wedge \mathsf{PCom.VrfyEval}(PC_z, 0, 0, w_z^0) = 1$ **then**
33: **if** $\mathsf{VCom.Vrfy}(VC_s, i, g^{s_i}, \pi_s^i) = 1$ **then**
34: **if** all $\ell+2$ elements in V_i are validated by Π_i **then**
35: extract PC_{dsk}, VC_{dpk} and $\{PC^k_\phi\}_{k\in[\ell]}$ from $\mathsf{Proof}^i_{\mathsf{rec}}$
36: $m_d \leftarrow \mathsf{ID}||d||g^s||PC_s||VC_s||PC_{dsk}||VC_{dpk}||\{PC^k_\phi\}_{k\in[\ell]}$
37: $\sigma^*_{\mathsf{r},d,i} \leftarrow \mathsf{TS.SignShare}_{n-t}(\textsc{ready}||m_d, tsk_i)$
38: send $\langle \textsc{Echo}, \mathsf{ID}, \sigma^*_{\mathsf{r},d,i}\rangle$ to $\mathcal{P}_d$ ▷ echo
39: **wait** until receive $\langle \textsc{Ready}, \mathsf{ID}, m'_d, \sigma'_{\mathsf{r},d}\rangle$ from $\mathcal{P}_d$
40: **if** $m_d = m'_d \wedge \mathsf{TS.Vrfy}_{n-t}(\textsc{ready}||m'_d, \sigma'_{\mathsf{r},d})$ **then**
41: $\sigma^*_{\mathsf{f},d,i} \leftarrow \mathsf{TS.SignShare}_{n-t}(\textsc{finish}||m_d, tsk_i)$
42: send $\langle \textsc{Finish}, \mathsf{ID}, \sigma^*_{\mathsf{f},d,i}\rangle$ to $\mathcal{P}_d$ ▷ finish
43: **output** $\langle V_i, \Pi_i\rangle$ ▷ output, for all nodes

44: **function** $\mathsf{GenRecPoly}[\langle \mathsf{ID}, d\rangle](t,n)$ ▷ cf. Alg. 5 for DPRF
45: $\langle dsk, dpk, PC_{dsk}, VC_{dpk}, \{dsk_i, dpk_i, w^i_{dsk}, \pi^i_{dpk}\}_{i\in[n]}\rangle \leftarrow \mathsf{DPRF.Init}(t,n)$
46: compute $y_i \leftarrow \mathcal{H}(\mathsf{DPRF.Eval}(\mathsf{ID}||d||i, dsk))$ for each $i \in [n]$
47: **for** $k \in [\ell]$ **do**
48: $I \leftarrow [(k-1)t+1, kt]$
49: $\phi_k(x) \leftarrow \mathsf{VSS.MakeSecret}(t, I, \langle y_i\rangle_{i\in I})$
50: $PC^k_\phi \leftarrow \mathsf{PCom.Commit}(\phi_k(x))$
51: **for** $i \in [n]$ **do**
52: $\langle \phi_k(i), w^{k,i}_\phi\rangle \leftarrow \mathsf{PCom.Witness}(\phi_k(x), i)$ for each $k \in [\ell]$
53: $\mathsf{Proof}^i_{\mathsf{rec}} \leftarrow \langle dpk_i, PC_{dsk}, VC_{dpk}, w^i_{dsk}, \pi^i_{dpk}, \{PC^k_\phi, w^{k,i}_\phi\}_{k\in[\ell]}\rangle$
54: **return** $\langle dsk_i, \{\phi_k(i)\}_{k\in[\ell]}, \mathsf{Proof}^i_{\mathsf{rec}}\rangle_{i\in[n]}$

VSS.Share, along with their evaluation proofs. The sharing polynomial is denoted as $f(x)$, satisfying $f(0)=s$ and $f(i)=s_i$ for every $i \in [n]$. Although VSS.Share guarantees the consistency of shares, it does not prevent the dealer from sharing an invalid input. To address this problem, we require $\mathcal{P}_d$ to commit to a zero polynomial $z(x) = f(x) - s$ (lines 5–7), ensuring that the dealer inputs the correct share instead of a garbage value. This step is crucial for the DPSS.Handoff procedure (cf. Algorithm 3). Next, $\mathcal{P}_d$ invokes GenRecPoly (line 8) to initialize DPRF key pairs (line 45) and $\ell = \lceil n/t \rceil$ recovery polynomials $\{\phi_k(x)\}_{k\in[\ell]}$ (lines 46–49), where each recovery polynomial goes through t DPRF evaluations. The DPRF key pairs and the recovery polynomials are accompanied by commitments, witnesses, and opening proofs, included in $\mathsf{Proof}^i_{\mathsf{rec}}$ (line 53). Then, $\mathcal{P}_d$ constructs a vector $\boldsymbol{v}_s = \{g^{s_i}\}_{i\in[n]}$ and commits to it by a vector commitment VC_s (lines 9–10). Finally, $\mathcal{P}_d$ generates a SHARE message for each $\mathcal{P}_i \in \mathcal{U}$, which includes a value set V_i containing $\ell+2$ values: a secret share s_i, a DPRF secret key share dsk_i, and ℓ polynomial evaluations $\{\phi_k(i)\}_{k\in[\ell]}$ (line 13), along with a proof set Π_i certifying the validity of V_i (line 14).

2. Echo. The second phase is triggered by $\mathcal{P}_i \in \mathcal{U}$ (lines 29–38). Upon receiving $\langle \textsc{Share}, \mathsf{ID}, V_i, \Pi_i\rangle$ from $\mathcal{P}_d$, node $\mathcal{P}_i$ first verifies this message, which includes verifying the correctness of sharing polynomial (line 32) and the opening proof of commitment vector $\boldsymbol{v}_s$ (line 33). The $\ell+2$ elements in V_i are implicitly validated by Π_i in line 34. If the message is valid, $\mathcal{P}_i$ replies to $\mathcal{P}_d$ with a signature share $\sigma^*_{\mathsf{r},d,i}$ on $\textsc{ready}||m_d$ via an ECHO message (lines 35–38), where m_d contains the commitments disseminated by the dealer (line 36). This signature share prevents $\mathcal{P}_d$ from sending inconsistent messages to different nodes.

3. Ready. In the third phase, $\mathcal{P}_d$ waits to collect $n-t$ valid signature shares (ECHO messages) from the nodes in $\mathcal{U}$, and then combines them into a full signature $\sigma_{\mathsf{r},d}$ (lines 16–21). Afterward, it multicasts the signature to all nodes via a READY message (line 22).

4. Finish. In this final phase, upon receiving a valid READY message from $\mathcal{P}_d$ (lines 39–40), node $\mathcal{P}_i$ replies to $\mathcal{P}_d$ with a signature share $\sigma^*_{\mathsf{f},d,i}$ via a FINISH message (line 42) and simultaneously outputs a value-proof tuple $\langle V_i, \Pi_i\rangle$. The dealer $\mathcal{P}_d$ waits to collect $n-t$ valid FINISH messages and combines them into a full signature $\sigma_{\mathsf{f},d}$ (lines 23–27). Once $\mathcal{P}_d$ obtains $\sigma_{\mathsf{f},d}$, it outputs a proof of sharing $\langle m_d, \sigma_{\mathsf{f},d}\rangle$.

Recovery. If node $\mathcal{P}_i$ detects the absence of its deserved share, it invokes this sub-protocol to request a share recovery. The absence of shares may be caused by network delay or dealer's misbehavior. Algorithm 2 provides the procedures for a batched recovery, where a node $\mathcal{P}_i$ recovers a set of shares generated by different dealers. When discussing the recovery of a share from wpACSS.Share$[\langle \mathsf{ID}, d\rangle]$, we assume that a proof of sharing exists. The detailed procedures of wpACSS.BatchedRecover are outlined below.

Algorithm 2 wpACSS.BatchedRecover$[\langle \mathsf{ID}, i\rangle]$

Let $\mathcal{H}: \mathbb{G} \to \mathbb{Z}_q$ be a hash function

1: **upon** receiving S^i_{help} as input **do** ▷ every $\mathcal{P}_i$ holds a help set S^i_{help}
2: multicast $\langle \textsc{Call}, \langle \mathsf{ID}, i\rangle, S^i_{\mathsf{help}}\rangle$ ▷ call help
3: **upon** receiving $\langle \textsc{Call}, \langle \mathsf{ID}, j\rangle, S^j_{\mathsf{help}}\rangle$ from $\mathcal{P}_j$ for the first time **do**
4: **for** each $d \in S^j_{\mathsf{help}}$ **do**
5: **if** have output $\langle V_{d,i}, \Pi_{d,i}\rangle$ from wpACSS.Share$[\langle \mathsf{ID}, d\rangle]$ **then**
6: $\mathsf{cont}_{d,i} \leftarrow \mathsf{RecContrib}[\langle \mathsf{ID}, d\rangle](j, V_{d,i}, \Pi_{d,i})$
7: $\mathsf{res}_i \leftarrow \mathsf{res}_i \cup \langle d, \mathsf{cont}_{d,i}\rangle$
8: **else**
9: $\mathsf{res}_i \leftarrow \mathsf{res}_i \cup \langle d, \perp\rangle$
10: send $\langle \textsc{Help}, \langle \mathsf{ID}, j\rangle, \mathsf{res}_i\rangle$ to $\mathcal{P}_j$ ▷ help
11: **upon** receiving $\langle \textsc{Help}, \langle \mathsf{ID}, i\rangle, \mathsf{res}_j\rangle$ from $\mathcal{P}_j$ for the first time **do**
12: parse res_j as $\langle d, \mathsf{cont}_{d,j}\rangle_{d \in S^i_{\mathsf{help}}}$ and verify each non-empty $\mathsf{cont}_{d,j}$
13: **for** each $d \in S^i_{\mathsf{help}}$ **do**
14: **wait** $t+1$ valid $\langle d, \mathsf{cont}_{d,j}\rangle$ in distinct res_j with same $\langle VC^d_{dpk}, PC^d_{\mathsf{mask}}\rangle$
15: denote the index set as I_d
16: extract $\langle s^m_{d,j}, F_{d,j}(i), \pi^{d,j}_{\mathsf{dprf}}\rangle$ from $\mathsf{cont}_{d,j}$, where $j \in I_d$
17: interpolate $s^m_{d,i}$ from $\{s^m_{d,j}\}_{j \in I_d}$
18: $F_d(i) \leftarrow \mathcal{H}(\mathsf{DPRF.Combine}(t, \mathsf{ID}||d||i, \{F_{d,j}(i), \pi^{d,j}_{\mathsf{dprf}}\}_{j \in I_d}))$
19: $s_{d,i} \leftarrow s^m_{d,i} - F_d(i)$ ▷ recover
20: **output** $\{s_{d,i}\}_{d \in S^i_{\mathsf{help}}}$

21: **function** RecContrib$[\langle \mathsf{ID}, d\rangle](j, V_i, \Pi_i)$ ▷ cf. Alg. 5 for DPRF
22: parse V_i as $\langle s_i, dsk_i, \{\phi_k(i)\}_{k \in [\ell]}\rangle$
23: parse Π_i as $\langle g^s, PC_s, w^i_s, PC_z, w^0_z, VC_s, \pi^i_s, \mathsf{Proof}^i_{\mathsf{rec}}\rangle$
24: parse $\mathsf{Proof}^i_{\mathsf{rec}}$ as $\langle dpk_i, PC_{dsk}, w^i_{dsk}, \pi^i_{dpk}, \{PC^k_\phi, w^{k,i}_\phi\}_{k \in [\ell]}\rangle$
25: $s^m_i \leftarrow s_i + \phi_{\lceil j/t \rceil}(i)$
26: $w^i_{\mathsf{mask}} \leftarrow w^i_s w^{\lceil j/t \rceil, i}_\phi$
27: $PC_{\mathsf{mask}} \leftarrow PC_s PC^{\lceil j/t \rceil}_\phi$
28: $\langle F_i(j), \mathsf{Proof}^i_{\mathsf{dprf}}\rangle \leftarrow \mathsf{DPRF.Contrib}(\mathsf{ID}||d||j, dsk_i, dpk_i, \pi^i_{dpk})$
29: $\mathsf{Proof}^i_{\mathsf{help}} \leftarrow \langle g^s, VC_{dpk}, PC_{\mathsf{mask}}, w^i_{\mathsf{mask}}, \mathsf{Proof}^i_{\mathsf{dprf}}\rangle$
30: **return** $\mathsf{cont}_i \leftarrow \langle s^m_i, F_i(j), \mathsf{Proof}^i_{\mathsf{help}}\rangle$

1. Call help. Node $\mathcal{P}_i$ first identifies a help set S^i_{help}, which includes the indexes of dealers from whom $\mathcal{P}_i$ has not received its deserved shares. $\mathcal{P}_i$ then multicasts the help set via a CALL message (lines 1–2). When S^i_{help} contains only one index d, Algorithm 2 is reduced to a non-batched version, where $\mathcal{P}_i$ aims to recover its share disseminated by the only dealer $\mathcal{P}_d$.

2. Help. When node $\mathcal{P}_i$ receives the help set S^j_{help} from $\mathcal{P}_j$, it checks whether it has output the value-proof tuple $\langle V_{d,i}, \Pi_{d,i}\rangle$ from wpACSS.Share$[\langle \mathsf{ID}, d\rangle]$ for each $d \in S^j_{\mathsf{help}}$ (lines 4–5). If so, it computes the response content $\mathsf{cont}_{d,i}$ using the RecContrib function (line 6). Otherwise, $\mathcal{P}_i$ sets $\mathsf{cont}_{d,i}$ as $\perp$ (line 9). After gathering all $\mathsf{cont}_{d,i}$ values into res_i, $\mathcal{P}_i$ sends res_i to $\mathcal{P}_j$ within a HELP message (line 10).

3. Recover. When the caller $\mathcal{P}_i$ receives a HELP message from $\mathcal{P}_j$ (line 11), it first parses res_j and verifies each non-empty content $\mathsf{cont}_{d,j}$, where $d \in S^i_{\mathsf{help}}$ (line 12). For each d, $\mathcal{P}_i$ waits for $t+1$ valid contents containing the same commitments $\langle VC^d_{dpk}, PC^d_{\mathsf{mask}}\rangle$ (lines 13–14). This ensures that the dealer $\mathcal{P}_d$ has sent consistent DPRF key shares and evaluations of recovery polynomials in wpACSS.Share$[\langle \mathsf{ID}, d\rangle]$. Then, it interpolates the i-th masked share $s^m_{d,i}$ and computes the DPRF evaluation $F_d(i)$ (lines 17–18). Finally, $\mathcal{P}_i$ recovers its share $s_{d,i} = s^m_{d,i} - F_d(i)$ (line 19).

6.3 Analysis of our wpACSS Protocol

Performance of provable sharing. In Algorithm 1, each message typically includes a constant number of evaluations and proofs, so the size of each message is $O(\lambda + |\pi|)$ bits, where $|\pi|$ represents the length of a proof. If we use KZG [41] and Pointproofs [35] for PCom and VCom, respectively, we have $|\pi| = O(\lambda)$. In this way, the communication complexity of our wpACSS.Share is $O(n(\lambda + |\pi|)) = O(\lambda n)$. Besides, as depicted by Figure 1, a provable sharing instance consumes four rounds of communication.

Performance of batched recovery. In Algorithm 2, when a node requests to recover $|S_{\mathsf{help}}|$ shares, there is one CALL message and n HELP messages, leading to an $O(n)$ message complexity. The CALL message is of $O(|S_{\mathsf{help}}|)$ bits, and every HELP message consumes $O(\lambda|S_{\mathsf{help}}|)$ bits, if KZG and Pointproofs are employed. Therefore, the overall communication complexity is $O(\lambda n|S_{\mathsf{help}}|)$. Besides, a batched recovery instance consumes two communication rounds.

Security analysis. We defer the detailed security proofs to Appendix C.

7 Our DPSS Protocol

Building upon wpACSS, we construct our DPSS.Handoff protocol in Algorithm 3, which invokes Algorithm 4 as a sub-protocol. For simplicity, we denote $\mathcal{P}^{e-1}_i$ as $\mathcal{P}_i$ and $\mathcal{P}^e_i$ as $\mathcal{P}'_i$. The i-th old and new shares are referred to as s_i and s'_i, respectively. We also let $t_{e-1} = t$, $t_e = t'$, $n_{e-1} = n$, $n_e = n'$, and $\mathsf{ID} = e$.

High-level overview. As shown in Figure 2, our DPSS.Handoff begins with n parallel wpACSS.Share instances, where each old node reshares its share with the new committee. Once the resharing is completed, the old node outputs a valid proof of sharing, which is then multicast to the new committee. Every new node waits for $t+1$ valid proofs and inputs them into MVBA. Then, the MVBA component selects $t+1$ valid resharing instances, and the new nodes compute their shares from the outputs of these instances. If an honest node has not yet output from any selected resharing instances, it invokes wpACSS.BatchedRecover to recover the missing shares. The missing of share may be caused by malicious dealers or network delay. To prevent old nodes from invoking wpACSS.Share with invalid old shares, the old committee transfers a commitment vector to the new committee before resharing, where the vector contains the commitments to all old shares. This vector is generated in a two-path sub-protocol GetNewCom (Algorithm 4) at the end of handoff, after the new nodes have obtained the new shares.

7.1 Details of our DPSS Protocol

Our DPSS.Handoff protocol, as shown in Algorithm 3, comprises five logical phases: *Transfer commitments*, *Reshare*, *Select*, *Recover*, and *Compute*, where the *Recover* phase is optional. The specific procedures are as follows.

1. Transfer commitments. Initially, each honest old node $\mathcal{P}_i$ checks whether it has an old share s_i and $t+1$ valid signatures on the commitment vector $\boldsymbol{v}_{\mathsf{old}} = \{g^{s_1}, ..., g^{s_n}\}$ containing the commitments to all old shares. If so, $\mathcal{P}_i$ commits to this vector by VC_{old} and generates an opening proof π^i_{old} for the i-th element g^{s_i}. Subsequently, $\mathcal{P}_i$ multicasts $\langle VC_{\mathsf{old}}, g^{s_i}, \pi^i_{\mathsf{old}}\rangle$ to the new committee $\mathcal{U}'$ using a COM message (lines 1–5). Each new node $\mathcal{P}'_i$ waits for $t+1$ valid COM messages from distinct old nodes, all with the same commitment VC_{old} (line 9). Then, $\mathcal{P}'_i$ uses the received elements to interpolate g^s and the vector $\boldsymbol{v}_{\mathsf{old}} = \{g^{s_1}, ..., g^{s_n}\}$ (line 10), where g^s is the commitment to the original secret s.

2. Reshare. Each old node $\mathcal{P}_i$ reshares its old share s_i to the new committee $\mathcal{U}'$ by wpACSS.Share$[\langle \mathsf{ID}, i\rangle]$, from which $\mathcal{P}_i$ outputs a proof of sharing $\langle m_i, \sigma_i\rangle$ (line 6). $\mathcal{P}_i$ multicasts this proof to $\mathcal{U}'$ via PROOF messages (line 7). Afterward, $\mathcal{P}_i$ deletes all private information and halts.

3. Select. New node $\mathcal{P}'_i$ awaits $t+1$ valid PROOF messages from distinct old nodes and records the $t+1$ proofs of sharing in W_i (lines 11–14). Then, $\mathcal{P}'_i$ inputs W_i into MVBA[ID] (lines 15–16) and waits for the output $\overline{W}$ from MVBA[ID] (line 17). The indexes of selected resharing instances are included in $I = \{j \mid \langle \bar{m}_j, \bar{\sigma}_j\rangle \in \overline{W} \wedge \langle \bar{m}_j, \bar{\sigma}_j\rangle \neq \perp\}$ (line 18).

4. Recover (optional). Upon obtaining the index set I, each new node $\mathcal{P}'_i$ identifies a help set S^i_{help} containing the indexes of all missing shares (lines 19–21). In case the help set is non-empty, i.e., there is any missing share, $\mathcal{P}'_i$ invokes wpACSS.BatchedRecover$[\langle \mathsf{ID}, i\rangle]$ (line 23). This phase is not executed in the optimistic case.

5. Compute. When $\mathcal{P}'_i$ obtains the shares $s_{j,i}$ for all $j \in I$, it interpolates the refreshed share s'_i from $\{(j, s_{j,i})\}_{j\in I}$ (line 24). Subsequently, each $\mathcal{P}'_i$ invokes GetNewCom[ID] (Algorithm 4) to obtain the new commitment vector $\boldsymbol{v}_{\mathsf{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$ (line 25). Then, $\mathcal{P}'_i$ signs $\boldsymbol{v}_{\mathsf{new}}$ and multicast the signature. Finally, $\mathcal{P}'_i$ waits for $t'+1$ valid and consistent signatures on $\boldsymbol{v}_{\mathsf{new}}$ and outputs $\langle s'_i, \boldsymbol{v}_{\mathsf{new}}\rangle$ (lines 26–28).

The GetNewCom[ID] sub-protocol consists of two paths: the optimistic and pessimistic paths. Every new node initially follows the optimistic path to generate the new commitment vector. If any misbehavior is detected, it multicasts an ERROR message and transitions to the pessimistic path. An honest node will also switch to the pessimistic path upon receiving an ERROR message. In the pessimistic path, every honest node that multicasts ERROR attempts to collect the commitments to subshares and interpolates them to obtain new commitments to the refreshed shares. When there are $t'+1$ new commitments, it interpolates the whole commitment vector $\boldsymbol{v}_{\mathsf{new}}$. To ensure correctness, the subshare commitments in our protocol are verifiable, and the proofs are extracted from the output of the wpACSS.Share sub-protocol. The step-by-step procedures for the two paths are outlined below (cf. Algorithm 4).

Algorithm 3 DPSS.Handoff[ID] between $\mathcal{U}$ and $\mathcal{U}'$

Let $\mathcal{U}$ be the old committee, with n nodes and t corrupted nodes, and $\mathcal{U}'$ be the new committee, with n' nodes and t' corrupted nodes
Let $\boldsymbol{v}_{\mathsf{old}} = \{g^{s_1}, ..., g^{s_n}\}$ include the commitments to all old shares, and g^s be the commitment to the secret s
Let the predicate of MVBA be: $P_{\mathsf{MVBA}}[\{\langle m_1, \sigma_1\rangle, \cdots, \langle m_n, \sigma_n\rangle\}] \equiv$ (there are $t+1$ valid signatures, and for each g^{s_j} extracted from m_j, it holds that $g^{s_j} = \boldsymbol{v}_{\mathsf{old}}[j]$)

As an old node $\mathcal{P}_i \in \mathcal{U}$:
1: **upon** receiving START **do**
2: **if** $\mathcal{P}_i$ have s_i and $t+1$ valid signatures on $\boldsymbol{v}_{\mathsf{old}}$ **then**
3: $VC_{\mathsf{old}} \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}_{\mathsf{old}})$
4: $\langle g^{s_i}, \pi^i_{\mathsf{old}}\rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}_{\mathsf{old}}, i)$
5: multicast $\langle \text{COM}, \mathsf{ID}, VC_{\mathsf{old}}, g^{s_i}, \pi^i_{\mathsf{old}}\rangle$ to $\mathcal{U}'$ ▷ transfer commitments
6: $\langle m_i, \sigma_i\rangle \leftarrow \mathsf{wpACSS.Share}[\langle \mathsf{ID}, i\rangle](\mathcal{U}', t', n', s_i)$ ▷ reshare
7: multicast $\langle \text{PROOF}, \mathsf{ID}, m_i, \sigma_i\rangle$ to $\mathcal{U}'$
8: delete all private information and halt ▷ halt (old node)

As a new node $\mathcal{P}'_i \in \mathcal{U}'$:
Init: start wpACSS.Share$[\langle \mathsf{ID}, j\rangle]$ for all $j \in [n]$; ▷ reshare
$W_i \leftarrow \{\langle m_j, \sigma_j\rangle_{j\in[n]}\}$, where $\langle m_j, \sigma_j\rangle \leftarrow \langle \perp, \perp\rangle$; $S^i_{\mathsf{help}} \leftarrow \emptyset$;
▷ transfer commitments
9: **upon** receiving $t+1$ valid $\langle \text{COM}, \mathsf{ID}, VC_{\mathsf{old}}, g^{s_j}, \pi^j_{\mathsf{old}}\rangle$ with VC_{old} **do**
10: interpolate g^s and $\boldsymbol{v}_{\mathsf{old}} = \{g^{s_1}, ..., g^{s_n}\}$

11: **upon** receiving $\langle \text{PROOF}, \mathsf{ID}, m'_j, \sigma'_j\rangle$ from $\mathcal{P}_j$ for the first time **do**
12: extract g^{s_j} from m'_j
13: **if** $\mathsf{TS.Vrfy}_{n'-t'}(\text{FINISH}||m'_j, \sigma'_j) = 1 \wedge g^{s_j} = \boldsymbol{v}_{\mathsf{old}}[j]$ **then**
14: $\langle m_j, \sigma_j\rangle \leftarrow \langle m'_j, \sigma'_j\rangle$, where $\langle m_j, \sigma_j\rangle \in W_i$
15: **if** there are $t+1$ non-empty elements in W_i **then**
16: input W_i to MVBA[ID] ▷ select

17: **upon** receiving $\overline{W} = \{\langle \bar{m}_j, \bar{\sigma}_j\rangle_{j\in[n]}\}$ from MVBA[ID] **do**
18: $I \leftarrow \{j \mid \langle \bar{m}_j, \bar{\sigma}_j\rangle \in \overline{W} \wedge \langle \bar{m}_j, \bar{\sigma}_j\rangle \neq \langle \perp, \perp\rangle\}$
19: **for** $j \in I$ **do**
20: **if** not yet receive $s_{j,i}$ from wpACSS.Share$[\langle \mathsf{ID}, j\rangle]$ **then**
21: $S^i_{\mathsf{help}} \leftarrow S^i_{\mathsf{help}} \cup \{j\}$
22: **if** $|S^i_{\mathsf{help}}| > 0$ **then** ▷ recover (optional)
23: $\{s_{j,i}\}_{j\in S^i_{\mathsf{help}}} \leftarrow \mathsf{wpACSS.BatchedRecover}[\langle \mathsf{ID}, i\rangle](S^i_{\mathsf{help}})$
24: interpolate s'_i from $\{(j, s_{j,i})\}_{j\in I}$ ▷ compute
25: $\boldsymbol{v}_{\mathsf{new}} \leftarrow \mathsf{GetNewCom}[\mathsf{ID}]$ ▷ cf. Alg. 4
26: sign $\boldsymbol{v}_{\mathsf{new}}$ and multicast the signature
27: **wait** for $t'+1$ valid and consistent signatures on $\boldsymbol{v}_{\mathsf{new}}$
28: **output** $\langle s'_i, \boldsymbol{v}_{\mathsf{new}}\rangle$ ▷ output (new node)

6. Optimistic path. Upon entering the GetNewCom[ID] sub-protocol, each new node $\mathcal{P}'_i$ initializes an empty index set I' and a flag $flg = 0$, and it multicasts the commitment $g^{s'_i}$ using NEWCOM messages. Then, $\mathcal{P}'_i$ awaits $t'+1$ distinct commitments and use them to interpolate $g^{s'_0}$ and $\boldsymbol{v}_{\mathsf{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$. If the interpolated $g^{s'_0}$ equals g^s, which is the commitment to the original share obtained in line 10 of Algorithm 3, $\mathcal{P}'_i$ returns the new commitment vector $\boldsymbol{v}_{\mathsf{new}}$. Otherwise, $\mathcal{P}'_i$ sets the flag as 1 and multicasts an ERROR message and enters the pessimistic path.

7. Pessimistic path. When node $\mathcal{P}'_i$ receives an ERROR

Algorithm 4 GetNewCom[ID], cf. Alg. 3 as main protocol

Init: $I' \leftarrow \emptyset$; $flg \leftarrow 0$; multicast $\langle \text{NEWCOM}, \text{ID}, g^{s'_i} \rangle$ to $\mathcal{U}'$

1: **upon** receiving $t'+1$ distinct $\langle \text{NEWCOM}, \text{ID}, g^{s'_j} \rangle$ **do** ▷ optimistic
2: interpolate $g^{s'_0}$ and $\boldsymbol{v}_{\text{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$
3: **if** $g^{s'_0} = g^{s}$ **then** ▷ g^{s} is obtained in line 10, Alg. 3
4: **return** $\boldsymbol{v}_{\text{new}}$ ▷ output
5: **else** $flg \leftarrow 1$; multicast $\langle \text{ERROR}, \text{ID} \rangle$ to $\mathcal{U}'$

6: **upon** receiving $\langle \text{ERROR}, \text{ID} \rangle$ from $\mathcal{P}'_j$ for the first time **do** ▷ pessimistic
7: $I_i \leftarrow I \backslash \mathcal{S}^i_{\text{help}}$ ▷ I and $\mathcal{S}^i_{\text{help}}$ are from line 18 and 21, Alg. 3
8: **for** $k \in I_i$ **do**
9: denote the output of wpACSS.Share$[\langle \text{ID}, k \rangle]$ as $\langle V_{k,i}, \Pi_{k,i} \rangle$
10: extract $\langle g^{s_{k,i}}, VC^k_s, \pi^{k,i}_s \rangle$ from $\Pi_{k,i}$
11: send $\langle \text{AUX}, \text{ID}, \{k, g^{s_{k,i}}, VC^k_s, \pi^{k,i}_s\}_{k \in I_i} \rangle$ to $\mathcal{P}'_j$

12: **upon** receiving $\langle \text{AUX}, \text{ID}, \{k, g^{s_{k,j}}, VC^k_s, \pi^{k,j}_s\}_{k \in I_j} \rangle$ from $\mathcal{P}'_j$ **do**
13: **if** $flg = 1 \wedge \text{VCom.Vrfy}(VC^k_s, k, g^{s_{k,j}}, \pi^{k,j}_s) = 1, \forall k \in I_j$ **then**
14: record $\langle k, g^{s_{k,j}} \rangle_{k \in I_j}$
15: **if** $I_j \neq I$ **then**
16: **for** $k \in I \backslash I_j$ **do** ▷ $I \backslash I_j = \mathcal{S}^j_{\text{help}}$
17: **wait** $t'+1$ AUX messages containing $g^{s_{k,*}}$ to interpolate $g^{s_{k,j}}$
18: record $\langle k, g^{s_{k,j}} \rangle$
19: interpolate $g^{s'_j} = g^{s_{0,j}}$ from $\langle k, g^{s_{k,j}} \rangle_{k \in I}$
20: $I' \leftarrow I' \cup \{j\}$
21: **if** $|I'| = t'+1$ **then**
22: interpolate $\boldsymbol{v}_{\text{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$ from $\{g^{s'_j}\}_{j \in I'}$
23: **return** $\boldsymbol{v}_{\text{new}}$ ▷ output

message from $\mathcal{P}'_j$, it replies with an AUX message containing the commitments to subshares and detailed verifiable information, which is extracted from the output of provable sharing instances. Specifically, let $I_i = I \backslash \mathcal{S}^i_{\text{help}}$ be an index set, such that for each $k \in I_i$, $\mathcal{P}'_i$ has output $\langle V_{k,i}, \Pi_{k,i} \rangle$ from wpACSS.Share$[\langle \text{ID}, k \rangle]$. $\mathcal{P}'_i$ extracts the tuples $\{k, g^{s_{k,i}}, VC^k_s, \pi^{k,i}_s\}_{k \in I_i}$ from $\{\Pi_{k,i}\}_{k \in I_i}$ and gathers them into the AUX message. On the other side, if $\mathcal{P}'_i$ has previously sent the ERROR message, then upon receiving an AUX message from $\mathcal{P}'_j$, $\mathcal{P}'_i$ verifies the commitments $g^{s_{k,j}}$ for all $k \in I_j$. If all verifications return true, $\mathcal{P}'_i$ records $\langle k, g^{s_{k,j}} \rangle_{k \in I_j}$. If $I_j \neq I$, which means $\mathcal{P}'_j$ fails to contribute some of the subshare commitments, then for each $k \in I \backslash I_j$, $\mathcal{P}'_i$ waits for $t'+1$ commitments $\{g^{s_{k,*}}\}$ from other AUX messages to interpolate the missing $g^{s_{k,j}}$. Once $\mathcal{P}'_i$ obtains $\{g^{s_{k,j}}\}_{k \in I}$, it interpolates a new commitment $g^{s'_j}$. When there are $t'+1$ new commitments, $\mathcal{P}'_i$ interpolates $\boldsymbol{v}_{\text{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$ and returns $\boldsymbol{v}_{\text{new}}$.

7.2 Analysis of our DPSS Protocol

Message complexity. In the *Transfer commitments* phase, each old node sends n' COM messages, resulting in a message complexity of $O(n^2)$. In the *Reshare* phase, the message complexity of each wpACSS.Share instance is $O(n)$, and every old node multicasts a PROOF message to all new nodes, leading to a message complexity of $O(n^2)$. The *Select* phase only involves one MVBA instance. Since we use the MVBA protocol [48] with a message complexity of $O(n^2)$, the message complexity of this phase is also $O(n^2)$. In the optional *Recover* phase, the nodes may invoke up to $O(n)$ wpACSS.BatchedRecover instances, which cost $O(n^2)$ message complexity. In the *Compute* phase, either path in GetNewCom has a message complexity of $O(n^2)$. In summary, the message complexity of DPSS.Handoff is $O(n^2)$.

Communication complexity. The communication complexity of DPSS.Handoff can be broken down into several steps. Each old node first multicasts an $O(\lambda)$-bit COM message to the new committee. Then, as the dealer, it invokes a wpACSS.Share instance, consuming $O(\lambda n)$ bits. Next, the new committee runs an MVBA instance, with $O(\lambda n)$-sized inputs, resulting in an $O(\lambda n^2)$ communication cost. The subsequent steps are divided into two cases.

Optimistic case: In this situation, no share recovery is needed, and the new committee invokes GetNewCom to obtain a new commitment vector. In the optimistic path of GetNewCom, each node only needs to multicast an $O(\lambda)$-bit NEWCOM message and then locally compute the new vector. Hence, the overall optimistic-case communication complexity of DPSS.Handoff is $O(\lambda n^2)$.

Worst case: In the worst situation, there are two bottlenecks that incur $O(\lambda n^3)$ communication cost. Firstly, at most $O(n)$ batched recovery instances are invoked, and each new node needs to batch-recover $O(n)$ shares, resulting in an $O(\lambda n^3)$ communication complexity. Secondly, in the pessimistic path of GetNewCom, each node multicasts an $O(\lambda)$-bit ERROR message and an $O(\lambda n)$-bit AUX message, so the communication complexity of GetNewCom in the worst case also reaches $O(\lambda n^3)$. In total, the worst-case communication complexity of DPSS.Handoff is $O(\lambda n^3)$.

Time complexity. The *Transfer commitments*, *Reshare*, *Recover*, and *Compute* phases consume up to 1, 5, 2, and 3 rounds of communication, respectively. The *Select* phase costs $O(1)$ time complexity due to the MVBA instance [48]. Therefore, the overall time complexity of DPSS.Handoff is $O(1)$.

Security analysis. We defer detailed security proofs of our DPSS.Handoff to Appendix D.

8 Extensions

The DPSS protocol in Section 7 only supports refreshing the low-threshold shares of a single secret. In this section, we briefly introduce two variations of our DPSS, batched DPSS and high-threshold DPSS. Both variations employ the hyperinvertible matrix [10], referred to as H matrix hereafter, to derive batched randomness. Given n shares, at most t of which are determined by the adversary, the H matrix allows a node to extract $n-t$ uniformly random shares.

8.1 Batched DPSS

Batched DPSS allows a batch of shares to be refreshed simultaneously within a single execution of DPSS, amortizing the communication cost. Our batched DPSS (bDPSS) adopts the strategy of LongLive [63]. From a high level, given a batch size $B = O(n)$, we let the old committee jointly generate a batch of uniformly random values $\{r_k\}_{k\in[B]}$, such that each old node $\mathcal{P}_i$ and new node $\mathcal{P}'_i$ hold $\{r_{k,i}\}_{k\in[B]}$ and $\{r'_{k,i}\}_{k\in[B]}$, respectively, with $r_{k,i}$ and $r'_{k,i}$ being the i-th shares of the same r_k. Then, we let the old committee reconstruct $\{s_k + r_k\}_{k\in[B]}$ and send them to the new committee, where s_k is the k-th secret. Each new node $\mathcal{P}'_i$ locally computes its new share $s'_{k,i} = s_k + r_k - r'_{k,i}$.

More specifically, each old node locally samples $B/(n-2t)$ random values and shares them by wpACSS.Share. Then, $n-t$ nodes are selected by MVBA, such that for each selected node $\mathcal{P}_j$, every honest node $\mathcal{P}_i$ outputs the shares of the $B/(n-2t)$ random values sampled by $\mathcal{P}_j$. Assuming $B = b(n-2t)$, $\mathcal{P}_i$ receives b sets of shares, each set containing $n-t$ shares. Using H matrices, $\mathcal{P}_i$ extracts $n-2t$ uniformly random shares from every $n-t$ non-uniform shares. Therefore, the total number of random shares derived by $\mathcal{P}_i$ is $b(n-2t) = B$.

We highlight the primary differences between DPSS and bDPSS in Figure 8, Appendix E. The details of bDPSS are presented in Algorithm 8, Appendix E.

Performance. For a batch size B, the communication complexity of bDPSS is $O(\lambda nB)$ in the optimistic case, which implies an amortized communication complexity of $O(\lambda n)$ per secret. The message complexity is $O(nB)$, and the time complexity is $O(1)$.

8.2 High-Threshold DPSS

In some threshold cryptosystems, $2t+1$ secret shares are required to reconstruct the secret or generate a threshold full signature, where at most t nodes are corrupted. Such high-threshold schemes are widely used in BFT protocols [18, 38, 50]. To support dynamic membership in these applications, a high-threshold DPSS is needed.

In the following, we describe how to extend our DPSS to the high-threshold setting using a high-threshold asynchronous distributed key generation (hADKG) protocol. This technique has been mentioned by Das et al. [24].

At a high level, the old committee inputs the low-threshold old shares $\{s_{\mathsf{low},i}\}_{i\in[n]}$ to the handoff protocol, so that every new node $\mathcal{P}'_i$ acquires a low-threshold new share $s'_{\mathsf{low},i}$. Additionally, $\mathcal{P}'_i$ outputs a high-threshold zero-share $z(i)$ from hADKG, where $z(x)$ is a random polynomial of degree $2t'$ subject to $z(0) = 0$. The refreshed high-threshold share for $\mathcal{P}'_i$ is calculated as $s'_{\mathsf{high},i} = s'_{\mathsf{low},i} + z(i)$. The high-threshold shares $\{s'_{\mathsf{high},i}\}_{i\in[n']}$ are used for DPSS.Recon, whereas the input to the subsequent handoff is $\{s'_{\mathsf{low},i}\}_{i\in[n']}$.

Performance. The hADKG protocol by Das et al. [24] utilizes n parallel ACSS instances and n parallel asynchronous Byzantine binary agreement (ABA) instances. To reduce the overall message and communication complexities, we may replace the ACSS instances with our wpACSS. Besides, we also replace the ABA instances with an MVBA instance of $O(n|m| + \lambda n^2)$ communication complexity and $O(1)$ time complexity [48]. In this way, we achieve a high-threshold DPSS with $O(n^2)$ message complexity, $O(\lambda n^2)$ optimistic-case communication complexity, and $O(1)$ time complexity.

9 Discussion

9.1 Eliminate the Trusted Setup

In our DPSS protocol discussed in Section 7, we rely on a trusted setup in every epoch. This assumption is also essential in state-of-the-art protocols [40, 63] to initialize (1) the public parameters for the underlying commitment schemes (e.g., powers of tau for the KZG scheme) and (2) threshold signing keys for each new committee (as required by MVBA [48]). A straightforward solution to lift this strong assumption is to employ an ADKG protocol. The state-of-the-art ADKG protocol [27] incurs a communication complexity of $O(\lambda n^3)$, aligning with both LongLive [63] and DyCAPS [40]. However, the time complexity in the worst case grows to $O(\log n)$.

Although in theory, the time complexity of ADKG can be reduced to $O(1)$, the authors of LongLive [63] caution that such a construction may not be practically efficient. Regarding our DPSS, even if $O(1)$ time complexity is achieved, employing an ADKG protocol with $O(\lambda n^3)$ communication complexity still harms our optimistic-case performance. In the following, we carefully design two strategies to eliminate this strong assumption. We believe that these strategies also apply to the batched and high-threshold DPSS described in Section 8, and we leave the justification for future work.

Achieving one-time trusted setup with a read-only bulletin board. We first address the setup of public parameters for commitment schemes. With a read-only bulletin board, we can employ the one-time trusted setup to produce the powers of tau for all $n \in [N]$, where $N = O(n)$ is the maximum allowed committee size. Prior to each handoff, both old and new committees retrieve $O(\lambda n)$-sized public parameters from the bulletin board, incurring $O(\lambda n^2)$ communication cost.

Regarding the threshold keys, they serve two purposes in our DPSS: (1) Creating proofs of sharing through threshold signatures in wpACSS (Algorithm 1); (2) Generating common coins in MVBA (explained below). To avoid using threshold keys in the optimistic case, we take the following steps.

In wpACSS, we implement the approach by Das et al. [23] to substitute threshold signatures with multi-signatures, accompanied by succinct non-interactive arguments of knowledge (SNARK). Specifically, the dealer combines $2t+1$ standard digital signatures into one multi-signature and generates

a SNARK proof for this aggregation. With a constant-sized universal SNARK instantiation (e.g., Plonk [34] and Vampire [46]), the communication complexity of wpACSS remains at $O(\lambda n)$. Moreover, both Plonk and Vampire facilitate a one-time setup, where the size of public parameters scales linearly with the maximum committee size $N = O(n)$.

As for MVBA [48], we are inspired by Crain's ABA [22], which has the *good-case-coin-free* property. This property states that if all honest nodes input the same value to the ABA, then all honest nodes can output without invoking a common coin sub-protocol. Hence, we enhance MVBA [48] with a *good-case-no-coin* property, such that no randomness is needed in the optimistic case.

Specifically, the MVBA protocol [48] consists of three main phases: broadcast, leader election, and voting. The latter two phases involve invoking the common coin sub-protocol, with the voting phase essentially being an instance of ABA. To achieve the *good-case-no-coin* property, we make two key modifications. First, during the leader election phase, the common coin sub-protocol is modified to output the predetermined value $(e \mod n_e)$ upon its first invocation, ensuring that the first output incurs no cost. Second, in the voting phase, we employ Crain's ABA [22]. In the optimistic case, where all nodes are honest and the network operates synchronously, all nodes will receive the necessary messages from the selected leader. As a result, all nodes will vote for the selected leader in the ABA protocol with input 1. According to the *good-case-coin-free* property of Crain's ABA [22], the ABA protocol ensures agreement on this value without invoking the common coin sub-protocol. Consequently, the MVBA protocol outputs without requiring any common randomness in the optimistic case. This also implies that the *good-case-no-coin* MVBA does not need to employ threshold keys to implement the common coin in the optimistic case. As the ABA protocol [22] incurs a communication cost of $O(\lambda n^2)$ bits, the overall communication complexity of both MVBA and our DPSS remains at $O(\lambda n^2)$ in the optimistic case.

If the initial round of MVBA fails, the committee activates the hADKG protocol [24] to establish threshold keys, which are utilized to generate common coins for the subsequent rounds of MVBA. This fallback results in a communication complexity of $O(\lambda n^3)$ and a message complexity of $O(\lambda n^3)$, which does not surpass the worst-case performance of LongLive [63].

In our design, all honest nodes attempt to execute the *good-case-no-coin* MVBA using a predetermined value as the output of the common coin in the first round, without invoking the ADKG protocol. As analyzed above, they can achieve agreement in the optimistic case within just one round. In other cases, if an agreement is not reached in the first round, the nodes turn to the operation of MVBA [48], which is guaranteed to produce an agreement among the nodes. Therefore, our *good-case-no-coin* MVBA satisfies the agreement property. Besides, this protocol is ensured to terminate in either the first or subsequent rounds, thus implying the termination property of MVBA. As for the external validity of MVBA, it is preserved because our modification does not involve the global predicate.

Removing the trusted setup. In the absence of a bulletin board, an alternative approach is to adopt sublinear-sized commitment schemes and multi-signatures that do not require any trusted setup. Specifically, we employ Bulletproofs [15] for PCom, and Merkle trees for VCom. In wpACSS, the threshold signatures are replaced with $O(\lambda \log n)$-sized multi-signatures by Qiu and Tang [54]. The *good-case-no-coin* MVBA described above is still adopted. By incorporating all these trusted-setup-free components, we achieve an asynchronous DPSS protocol with a transparent setup. This protocol achieves an optimistic-case communication complexity of $O(\lambda n^2 \log n)$ and a worst-case communication complexity of $O(\lambda n^3)$. Similarly, the optimistic-case message complexity is $O(\lambda n^2)$, while the worst-case message complexity remains $O(\lambda n^3)$. Such performance still surpasses the existing asynchronous DPSS protocols.

9.2 Support Non-uniformly Random Secrets

Recall that our DPSS protocol discussed in Section 7 requires the secret to be uniformly random, while many typical applications of DPSS are not constrained by this uniformly random requirement, including: (1) Threshold cryptosystems, where secret keys are inherently uniformly random. (2) The "BMR Escape Hatch" described in [63], which uses uniformly random values as secrets. (3) Cloud storage of confidential files, such as in COBRA [58], where files can be encrypted, and the uniformly random encryption keys are protected using DPSS.

In this section, we further discuss how to extend our DPSS to protect a non-uniformly random secret s. At a high level, users can employ a threshold ElGamal encryption (TE) scheme to encrypt the non-uniform secret s while periodically refreshing the uniform encryption key using our proposed DPSS. The TE scheme is a variant of the standard ElGamal encryption algorithm, where the secret key is distributed among multiple nodes as secret shares. It consists of four algorithms: TE.KeyGen, TE.Enc, TE.DecShare, and TE.Dec. TE.KeyGen takes a threshold t, a committee size n, and a security parameter λ as inputs to generate a secret key esk, a public key $epk = g^{esk}$, and a set of key shares $\{esk_i, epk_i\}_{i\in[n]}$. TE.Enc encrypts a message m using epk, and the encrypted message is denoted as c. TE.DecShare computes a decryption share d_i from an encrypted message c and a secret key share esk_i. Finally, TE.Dec reconstructs the plaintext message m from at least $t+1$ valid decryption shares $\{d_i\}_{i\in I}$, where I is an index set satisfying $|I| = t+1$.

Specifically, we leverage the TE primitive to protect a non-uniform secret s. In epoch $e = 1$, the owner of s (the client) first invokes TS.KeyGen to uniformly samples a secret key esk, a public key $epk = g^{esk}$, and a set of key shares

$\{esk_i, epk_i\}_{i\in[n]}$. The key shares are then distributed among the nodes in $\mathcal{U}^1$ using DPSS.Share (cf. Algorithm 6, Appendix B.1). Meanwhile, the owner encrypts s using epk via TE.Enc, obtaining the ciphertext c, which is disseminated to $\mathcal{U}^1$ via a reliable broadcast protocol [5]. Upon receiving c, each node in $\mathcal{U}^1$ generates a threshold signature share on c and multicasts it to the other nodes in committee $\mathcal{U}^1$. The node then waits to collect at least t_1+1 valid signature shares, which can be combined to form a full signature. For any epoch $e>1$, the shares of esk are periodically refreshed using our proposed DPSS.Handoff protocol. Simultaneously, each old node multicasts the ciphertext c to the new committee, if it possesses a valid full signature. Once the new committee receives at least $t_{e-1}+1$ identical copies of c, each new node generates a new signature share on c with the updated threshold t_e and multicasts it to $\mathcal{U}^e$. The new node then collects at least t_e+1 valid signature shares and forms a full signature on c. Note that at the end of each handoff, every new node obtains a commitment vector $\boldsymbol{v}_{\mathsf{new}}$, which exactly includes all public key shares of the new nodes.

When retrieving s, there are two approaches. In the first approach, each node $\mathcal{P}_i^e$ in $\mathcal{U}^e$ generates a decryption share d_i via TE.DecShare. The owner collects t_e+1 decryption shares and inputs them into TE.Dec, thus obtaining the decrypted secret s as output. The second approach utilizes the DPSS.Recon (cf. Algorithm 7, Appendix B.2) to reconstruct the esk. During this process, the client implicitly obtains the public key epk and verifies esk against epk. The new committee then sends the valid ciphertext m to the client. Once the client receives at least t_e+1 identical copies of m, it uses esk to decrypt m and recovers s via the standard ElGamal decryption algorithm.

10 Performance Evaluation

We implemented our DPSS in Go, using kilic's bls12-381 curve library [42]. For PCom, we used the KZG scheme implemented by protolambda [53]. For VCom, we employed Pointproofs implemented by Zhang [64]. For threshold signatures, we used Boldyreva's scheme [13] implemented by drand/kyber [31]. Finally, for MVBA, we adopted the implementation of Dumbo-MVBA from [30], which is part of the Dory protocol [65].

We deployed our DPSS protocol on up to 254 Amazon EC2 t2.medium instances, each serving as a node. Each instance has 2 vCPUs and 4GB memory. The nodes were evenly distributed across eight regions: North Virginia, North California, Canada, London, Seoul, Singapore, Sydney, and Sao Paulo. During the experiments, we performed handoffs between two committees of the same size, with the committee size ranging from 4 to 127 nodes. In the following, the evaluation results are under the trusted setup assumption, unless otherwise stated.

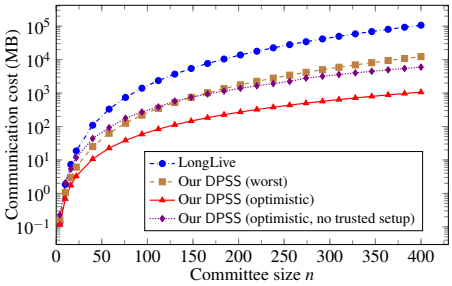

Figure 4: Estimated comm. cost of our DPSS and LongLive.

Communication cost. We evaluate the estimated concrete communication of our DPSS for $n \leq 400$. We also make a comparison with the state-of-the-art protocol LongLive [63]. To ensure a fair comparison, we employ the same MVBA implementation. Besides, the ACSS in LongLive is instantiated by hbACSS [62], with the KZG scheme as PCom.

We consider both optimistic and worst cases for our DPSS. In the optimistic case, the batched recovery is never invoked, and the new commitments are generated via the optimistic path. In the worst-case scenario, we require all nodes to execute share recovery for all dealers selected by MVBA, even if the nodes have already received shares from these dealers. Additionally, the nodes generate the new commitments through the pessimistic path.

As depicted in Figure 4, our DPSS outperforms LongLive in both optimistic and worst cases. Specifically, in the optimistic case, the communication overhead of our DPSS is a mere 1.0% to 5.2% of LongLive (i.e., about 19–100x smaller), while in the worst case, it is 7.1% to 12.7% (i.e., about 8–17x smaller). The advantage in the optimistic case becomes more pronounced as n scales, owing to the reduction of a $O(n)$ factor in communication complexity. When there is no trusted setup, the concrete communication cost of our DPSS outperforms LongLive (with per-epoch trusted setup) as early as $n > 13$ in the optimistic case.

Latency. The latency is measured as the average duration between the start time of an old node and the time that a new node outputs a refreshed share. We evaluated the latency of our DPSS in the optimistic case, crash case [1], and worst case, respectively. To further reveal the influence of computational cost, we implemented another version of our DPSS with Merkle tree [11] as VCom. Compared to the Pointproofs-based DPSS, this variant reduces the computational cost but increases the communication complexity from $O(\lambda n^2)$ to $O(\lambda n^2 \log n)$ in the optimistic case. Furthermore, we note that this approach cannot be applied directly to LongLive, as it employs Pedersen commitments as PCom but does not use vector commitments. Replacing Pedersen commitments with vector commitments would require a significant redesign of LongLive's construction.

As shown in Figure 5, we observe that our DPSS with Merkle trees as vector commitments is 10% to 20% faster than

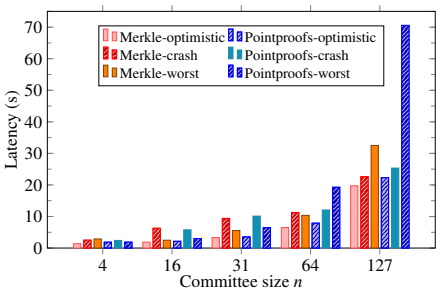

Figure 5: Latency of our DPSS in AWS.

the Pointproofs version. This difference is due to the Pointproofs scheme consuming more computational resources, and the advantage of constant-length proofs not being apparent. In the worst-case scenario, our DPSS takes about 30% to 210% more runtime than in the optimistic case. This increase is attributed to the frequent occurrence of share recovery. Especially noteworthy is the low latency when the committee consists of only 4 nodes, where the Merkle-tree version takes just 1.38 seconds and the Pointproofs version takes 1.87 seconds in the optimistic case. Even as the committee size scales up to 64, the latency for each optimistic-case handoff consistently stays below 8 seconds for both versions.

For the crash case, we designate t nodes within each committee to behave as dumb nodes at the start of the handoff, refraining from transmitting any information to others, while the remaining nodes are honest. In this situation, there is no invocation of share recovery instances. However, the honest nodes must wait longer to receive a sufficient number of messages during the handoff because the crashed nodes are not contributing to the communication. As illustrated in Figure 5, the impact of crashed nodes is more pronounced when the committee sizes are 16 and 31, where the crash-case latency is approximately 170% to 230% larger than that in the all-honest scenario. As the committee size reaches 64 and 127, the difference in latency decreases. This happens because the increasing computation time becomes the dominant factor as the committee size n grows, effectively masking the impact of network latency in the overall measurements.

Comparison of computational cost with LongLive. As the LongLive source code [61] supports only local benchmarks, the comparison was conducted on a single EC2 r5.large instance with 16GB memory. To align with the benchmark setting of LongLive, we limited the number of CPU cores to one. This experiment can be viewed as a comparison of computational costs, without accounting for network delays.

Considering that our DPSS and LongLive employ similar batching techniques (cf. Section 8.1), it is reasonable to focus on the single-secret scenario. We still assume all nodes are honest. The results are presented in Table 3. The local benchmarks demonstrate that both versions of our DPSS, based on Merkle trees and Pointproofs, respectively, outperform LongLive in terms of runtime. As n increases, the advantage of our DPSS becomes more pronounced. Notably, our Merkle-tree-based DPSS achieves a runtime reduction of approximately 50% starting from $n = 16$ when compared with LongLive. This performance difference indicates that LongLive experiences a faster increase in latency as n grows.

Table 3: Comparison of computational cost (seconds)

Scale	LongLive	DPSS-Merkle-tree		DPSS-Pointproofs	
$n = 10$	6.20	3.81	$\downarrow$ 38.5%	5.82	$\downarrow$ 6.20%
$n = 16$	19.80	10.36	$\downarrow$ 47.7%	15.83	$\downarrow$ 20.0%
$n = 31$	108.82	44.45	$\downarrow$ 59.1%	68.60	$\downarrow$ 37.0%
$n = 52$	426.21	148.74	$\downarrow$ 65.1%	239.65	$\downarrow$ 43.8%

Moreover, we highlight that the benefits of our $O(n^2)$ message complexity may not be fully evident in the above comparison. When deployed in a distributed environment, our DPSS is expected to exhibit even better performance than LongLive, showcasing its scalability and efficiency on a larger scale.

11 Conclusion

In this paper, we have designed and implemented an asynchronous DPSS protocol with $O(n^2)$ message complexity. In the optimistic case, we have further reduced the redundant communication costs of DPSS, while ensuring that our DPSS protocol does not compromise on worst-case performance compared to the state-of-the-art. We first discussed our DPSS with a per-epoch trusted setup assumption and then provided two strategies to remove it. Experimental results demonstrate a significant reduction in communication and computation costs for our DPSS in the optimistic case.

Our DPSS may benefit BFT-based blockchains by enabling the reconfiguration of shareholder committees. Besides, our Weak Provable ACSS could potentially enhance the optimistic-case performance of the state-of-the-art ADKG protocols [24,27]. We leave these explorations as future work.

References

[1] Ittai Abraham, Danny Dolev, Alon Kagan, and Gilad Stern. Brief announcement: Authenticated consensus in synchronous systems with mixed faults. In *DISC*, volume 246 of *LIPIcs*, pages 38:1–38:3, 2022.

[2] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. Asymptotically optimal validated asynchronous byzantine agreement. In *ACM PODC*, pages 337–346, 2019.

[3] Shashank Agrawal, Payman Mohassel, Pratyay Mukherjee, and Peter Rindal. DiSE: Distributed symmetric-key encryption. In *ACM CCS*, pages 1993–2010, 2018.

[4] Andreea B. Alexandru, Erica Blum, Jonathan Katz, and Julian Loss. State machine replication under changing

network conditions. In *ASIACRYPT*, volume 13791 of *LNCS*, pages 681–710. Springer, 2022.

[5] Nicolas Alhaddad, Sourav Das, Sisi Duan, Ling Ren, Mayank Varia, Zhuolun Xiang, and Haibin Zhang. Balanced byzantine reliable broadcast with near-optimal communication and improved computation. In *ACM PODC*, pages 399–417, 2022.

[6] Nicolas Alhaddad, Mayank Varia, and Haibin Zhang. High-threshold AVSS with optimal communication complexity. In *FC*, volume 12675 of *LNCS*, pages 479–498. Springer, 2021.

[7] Michael Backes, Amit Datta, and Aniket Kate. Asynchronous computational VSS with reduced communication complexity. In *CT-RSA*, volume 7779 of *LNCS*, pages 259–276. Springer, 2013.

[8] Soumya Basu, Alin Tomescu, Ittai Abraham, Dahlia Malkhi, Michael K. Reiter, and Emin Gün Sirer. Efficient verifiable secret sharing with share recovery in BFT protocols. In *ACM CCS*, pages 2387–2402, 2019.

[9] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *ACM STOC*, pages 503–513, 1990.

[10] Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure MPC with linear communication complexity. In *TCC*, volume 4948 of *LNCS*, pages 213–230. Springer, 2008.

[11] Cameron Bergoon. Merkle tree in golang, 2022. https://github.com/cbergoon/merkletree.

[12] G. R. Blakley. Safeguarding cryptographic keys. In *1979 International Workshop on Managing Requirements Knowledge*, pages 313–318. IEEE, 1979.

[13] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme. In *PKC*, volume 2567 of *LNCS*, pages 31–46. Springer, 2003.

[14] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.

[15] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Gregory Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE SP*, pages 315–334, 2018.

[16] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. Asynchronous verifiable secret sharing and proactive cryptosystems. In *ACM CCS*, pages 88–97, 2002.

[17] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. Secure and efficient asynchronous broadcast protocols. In *CRYPTO*, pages 524–541. Springer, 2001.

[18] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constantinople: practical asynchronous byzantine agreement using cryptography. In *ACM PODC*, 2000.

[19] Dario Catalano and Dario Fiore. Vector commitments and their applications. In *PKC*, volume 7778 of *LNCS*, pages 55–72. Springer, 2013.

[20] Ashish Choudhury, Martin Hirt, and Arpita Patra. Asynchronous multiparty computation with linear communication complexity. In *DISC*, volume 8205 of *LNCS*, pages 388–402. Springer, 2013.

[21] Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Trans. Inf. Theory*, 63(1):428–468, 2017.

[22] Tyler Crain. Experimental evaluation of asynchronous binary byzantine consensus algorithms with $t < n/3$ and $o(n^2)$ messages and $o(1)$ round expected termination. *arXiv preprint arXiv:2002.08765*, 2020.

[23] Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In *ACM CCS*, pages 356–370, 2023.

[24] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *USENIX Security*, pages 5359–5376, 2023.

[25] Sourav Das, Zhuolun Xiang, and Ling Ren. Asynchronous data dissemination and its applications. In *ACM CCS*, pages 2705–2721, 2021.

[26] Sourav Das, Zhuolun Xiang, and Ling Ren. Powers of tau in asynchrony. In *NDSS*, pages 1–17, 2024.

[27] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew K. Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. In *IEEE SP*, pages 2518–2534, 2022.

[28] Yvo Desmedt and Sushil Jajodia. Redistributing secret shares to new access structures and its applications. Technical report, Citeseer, 1997.

[29] Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. *JACM*, 32(1):191–204, 1985.

[30] DoryTeam. Dory BFT consensus, 2023. https://github.com/xygdys/Dory-BFT-Consensus.

[31] Drand. Drand/kyber library, 2023. https://github.com/drand/kyber.

[32] Sisi Duan, Xin Wang, and Haibin Zhang. Fin: practical signature-free asynchronous common subset in constant time. In *ACM CCS*, pages 815–829, 2023.

[33] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, pages 427–437. IEEE Computer Society, 1987.

[34] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR Cryptology ePrint Archive, 2019.

[35] Sergey Gorbunov, Leonid Reyzin, Hoeteck Wee, and Zhenfei Zhang. Pointproofs: Aggregating proofs for multiple vector commitments. In *ACM CCS*, pages 2007–2023, 2020.

[36] Vipul Goyal, Abhiram Kothapalli, Elisaweta Masserova, Bryan Parno, and Yifan Song. Storing and retrieving secrets on a blockchain. In *PKC*, volume 13177 of *LNCS*, pages 252–282. Springer, 2022.

[37] Bingyong Guo, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. Speeding dumbo: Pushing asynchronous BFT closer to practice. In *NDSS*, pages 1–18, 2022.

[38] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. Dumbo: Faster asynchronous BFT protocols. In *ACM CCS*, pages 803–818, 2020.

[39] Amir Herzberg, Stanisław Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. In *CRYPTO*, pages 339–352. Springer, 1995.

[40] Bin Hu, Zongyang Zhang, Han Chen, You Zhou, Huazu Jiang, and Jianwei Liu. DyCAPS: Asynchronous proactive secret sharing for dynamic committees. IACR Cryptology ePrint Archive, 2022.

[41] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT*, volume 6477 of *LNCS*, pages 177–194. Springer, 2010.

[42] kilic. BLS12-381 library, 2022. https://github.com/kilic/bls12-381.

[43] Eleftherios Kokoris-Kogias, Dahlia Malkhi, and Alexander Spiegelman. Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In *ACM CCS*, pages 1751–1767, 2020.

[44] Christophe Levrat, Matthieu Rambaud, and Antoine Urban. Breaking the $t < n/3$ consensus bound: Asynchronous dynamic proactive secret sharing under honest majority. IACR Cryptology ePrint Archive, 2023.

[45] Benoît Libert, Alain Passelègue, and Mahshid Riahinia. Pointproofs, revisited. In *ASIACRYPT*, volume 13794 of *LNCS*, pages 220–246. Springer, 2022.

[46] Helger Lipmaa, Janno Siim, and Michal Zajac. Counting vampires: From univariate sumcheck to updatable ZK-SNARK. In *ASIACRYPT*, volume 13792 of *LNCS*, pages 249–278. Springer, 2022.

[47] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. HoneyBadgerMPC and asynchromix: Practical asynchronous MPC and its application to anonymous communication. In *ACM CCS*, pages 887–903, 2019.

[48] Yuan Lu, Zhenliang Lu, Qiang Tang, and Guiling Wang. Dumbo-mvba: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In *ACM PODC*, pages 129–138, 2020.

[49] Sai Krishna Deepak Maram, Fan Zhang, Lun Wang, Andrew Low, Yupeng Zhang, Ari Juels, and Dawn Song. CHURP: dynamic-committee proactive secret sharing. In *ACM CCS*, pages 2369–2386, 2019.

[50] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of BFT protocols. In *ACM CCS*, pages 31–42, 2016.

[51] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and KDCs. In *EUROCRYPT*, volume 1592 of *LNCS*, pages 327–346. Springer, 1999.

[52] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO*, volume 576 of *LNCS*, pages 129–140. Springer, 1991.

[53] protolambda. Go-KZG library, 2023. https://github.com/protolambda/go-kzg.

[54] Tian Qiu and Qiang Tang. Predicate aggregate signatures and applications. In *ASIACRYPT*, volume 14439 of *LNCS*, pages 279–312. Springer, 2023.

[55] David A Schultz, Barbara Liskov, and Moses Liskov. Mobile proactive secret sharing. In *ACM PODC*, pages 458–458, 2008.

[56] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

[57] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: Dag bft protocols made practical. In *ACM CCS*, pages 2705–2718, 2022.

[58] Robin Vassantlal, Eduardo Alchieri, Bernardo Ferreira, and Alysson Bessani. Cobra: Dynamic proactive secret sharing for confidential BFT services. In *IEEE SP*, pages 1335–1353, 2022.

[59] Yunzhou Yan, Yu Xia, and Srinivas Devadas. Shanrang: Fully asynchronous proactive secret sharing with dynamic committees. IACR Cryptology ePrint Archive, 2022.

[60] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. Hotstuff: BFT consensus with linearity and responsiveness. In *ACM PODC*, pages 347–356, 2019.

[61] Thomas Yurek. DPSS library, 2022. https://github.com/tyurek/dpss.

[62] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. hbACSS: How to robustly share many secrets. In *NDSS*, 2022.

[63] Thomas Yurek, Zhuolun Xiang, Yu Xia, and Andrew Miller. Long live the honey badger: Robust asynchronous DPSS and its applications. In *USENIX Security*, pages 5413–5430, 2023.

[64] Zhenfei Zhang. Pointproof, 2022. https://github.com/zhenfeizhang/pointproofs.

[65] Zongyang Zhang, You Zhou, Sisi Duan, Haibin Zhang, Bin Hu, Licheng Wang, and Jianwei Liu. Dory: Faster asynchronous BFT with reduced communication for permissioned blockchains. IACR Cryptology ePrint Archive, 2023.

[66] Lidong Zhou, Fred B. Schneider, and Robbert van Renesse. APSS: proactive secret sharing in asynchronous systems. *ACM Trans. Inf. Syst. Secur.*, 8(3):259–286, 2005.

A Instantiations of Building Blocks

Polynomial commitment (PCom) [33, 41] allows a node commit to a polynomial and provide witnesses of its evaluations. We mainly use four algorithms: PCom.Setup, PCom.Commit, PCom.Witness, and PCom.VerifyEval.

- $pp \leftarrow \mathsf{PCom.Setup}(t, 1^\lambda)$: this algorithm initializes the public parameters. It takes as inputs a degree bound t and a security parameter λ in unary form. The output is public parameters pp. We sometimes omit pp for simplicity.
- $PC_\phi \leftarrow \mathsf{PCom.Commit}(\phi(x), pp)$: this algorithm commits to a polynomial. It takes as inputs a polynomial $\phi(x) \in \mathbb{Z}_p[x]$ and public parameters pp. The output is a commitment PC_ϕ.
- $\langle \phi(i), w^i \rangle \leftarrow \mathsf{PCom.Witness}(\phi(x), i, pp)$: this algorithm creates a witness for polynomial evaluation. It takes as inputs a polynomial $\phi(x)$, an index i, and public parameters pp. The output is an value $\phi(i)$ and a witness w^i.
- $0/1 \leftarrow \mathsf{PCom.VrfyEval}(PC_\phi, i, v, w^i, pp)$: this algorithm verifies a polynomial evaluation. It takes as inputs a commitment PC_ϕ, an index i, an evaluation v, a witness w^i, and public parameters pp. It outputs 1 iff $v = \phi(i)$.

We use the KZG scheme [41] for the polynomial commitment, due to its constant commitment and witness size. It has the following properties except with negligible probability.

- *Correctness.* For all honestly generated public parameters and a fixed commitment, the output of PCom.Witness always passes PCom.VrfyEval.
- *Degree Binding.* If t is the input to PCom.Setup and $t' > t$, an adversary cannot commit to a degree-t' polynomial.
- *Evaluation Binding.* Given a commitment PC_ϕ, an adversary cannot generate two witnesses, w^i and w'^i, that both pass PCom.VrfyEval and satisfy $\phi(i) \neq \phi'(i)$.
- *Hiding.* Given t evaluation-witness pairs $\langle i, \phi(i), w^i \rangle$ and the commitment PC_ϕ to a degree-t polynomial $\phi(x)$, an adversary cannot determine $\phi(i')$ for any unqueried i'.
- *Homomorphism.* The commitment to $\phi(x) = \phi_1(x) + \phi_2(x)$ can be computed as $PC_\phi = PC_{\phi_1} PC_{\phi_2}$.[4] Similarly, $w^i_\phi = w^i_{\phi_1} w^i_{\phi_2}$ holds for $\phi(i) = \phi_1(i) + \phi_2(i)$.

Verifiable secret sharing (VSS) [6, 52] allows a dealer to share a secret among n nodes, such that each node can verify the correctness of its share. We mainly use four algorithms: VSS.Share, VSS.Vrfy, VSS.Recon, and VSS.MakeSecret.

- $\{PC_s, \langle s_i, w^i_s \rangle_{i \in [n]}\} \leftarrow \mathsf{VSS.Share}(t, n, s, 1^\lambda)$: this algorithm generates secret shares. It takes as inputs a threshold t, a committee size n, a secret s, and a security parameter λ in unary form. The output is a polynomial commitment PC_s and a set of share-witness pairs.
- $0/1 \leftarrow \mathsf{VSS.Vrfy}(PC_s, i, s_i, w^i_s)$: this algorithm verifies a secret share. It takes as input a polynomial commitment PC_s, an index i, a secret share s_i and its witness w^i_s. It outputs 1 iff s_i is a valid share of s.

[4] Specially, in the context of the KZG [41] scheme, g^s is a polynomial commitment to $\phi_1(x) = s$. Hence, $PC_\phi = g^s PC_{\phi_2}$ can be seen as the commitment to $\phi(x) = s + \phi_2(x)$.

- $s/\perp \leftarrow \mathsf{VSS.Recon}(t, \{s_i\}_{i\in I})$: this algorithm reconstructs a secret. It takes as inputs a threshold t and a set of shares $\{s_i\}_{i\in I}$, where $|I| \geq t+1$. The output is a secret s or an empty symbol $\perp$.

- $\phi(x) \leftarrow \mathsf{VSS.MakeSecret}(t, I, \{y_i\}_{i\in I})$: this algorithm generates a polynomial from a set of points. It takes as inputs a threshold t, an index set I, and a set of values $\{y_i\}_{i\in I}$, where $|I| \leq t$. It samples $t+1-|I|$ random points and interpolates a degree-t polynomial $\phi(x)$ as output, satisfying that $\phi(i) = y_i, \forall i \in I$.

The above algorithms are all non-interactive, so they are oblivious to the network model. A VSS protocol has the following properties [41] except with negligible probability.

- *Secrecy.* An adversary learns nothing about the secret, given t shares and the public information.

- *Correctness.* If the dealer is honest, the output of VSS.Recon is the same as the input to VSS.Share. If the dealer misbehaves, the output of VSS.Recon is either the same as the input to VSS.Share or $\perp$.

- *Agreement.* Whenever two distinct nodes invoke VSS.Recon, they will always output the same value.

Vector commitment (VCom) [19, 35, 45] allows a node to commit to a vector and later open some of its elements. We mainly use four algorithms: VCom.Setup, VCom.Commit, VCom.Open, and VCom.Vrfy.

- $pp \leftarrow \mathsf{VCom.Setup}(n, 1^\lambda)$: this algorithm initializes the public parameters. It takes as inputs a vector size n and a security parameter λ in unary form. The output is public parameters pp. We sometimes omit pp for simplicity.

- $VC_{\boldsymbol{v}} \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}, pp)$: this algorithm commits to a vector. It takes as inputs a vector $\boldsymbol{v} = \{v_1, ..., v_n\}$ and public parameters pp. The output is a commitment $VC_{\boldsymbol{v}}$.

- $\langle v_i, \pi^i_{\boldsymbol{v}} \rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}, i, pp)$: this algorithm creates an opening proof for the i-th position in vector. It takes as inputs a vector $\boldsymbol{v}$, an index i, and public parameters pp. The output is an evaluation v_i and a proof $\pi^i_{\boldsymbol{v}}$.

- $0/1 \leftarrow \mathsf{VCom.Vrfy}(VC_{\boldsymbol{v}}, i, v_i, \pi^i_{\boldsymbol{v}}, pp)$: this algorithm verifies a opening proof. It takes as inputs a commitment $VC_{\boldsymbol{v}}$, an index i, a vector element v_i, a proof $\pi^i_{\boldsymbol{v}}$, and public parameters pp. It outputs 1 iff v_i is the i-th element of $\boldsymbol{v}$.

We use Pointproofs [35] for the vector commitment, due to its constant commitment size and proof size. It satisfies the following properties except with negligible probability.

- *Correctness.* For all honestly generated public parameters and a fixed vector commitment, the output of VCom.Open always passes VCom.Vrfy.

- *Binding.* Given a fixed vector commitment and public parameters, an adversary cannot generate two distinct vector-proof tuples that both pass VCom.Vrfy at the same position.

- *Hiding.* Given a commitment and several opening proofs of a vector, an adversary cannot obtain the information of any other elements in the vector.

Threshold signature (TS) [13] allows a quorum of nodes to jointly sign a message. It has five algorithms: TS.KeyGen, TS.SignShare, TS.VrfyShare, TS.Combine, and TS.Vrfy.

- $\{tpk, \langle tpk_i, tsk_i \rangle_{i\in[n]}\} \leftarrow \mathsf{TS.KeyGen}(t, n, 1^\lambda)$: this algorithm generates threshold signing key pairs. It takes as inputs a threshold t, a committee size n, and a security parameter λ in unary form. The output is a threshold public key tpk, a threshold public key share set $\{tpk_i\}_{i\in[n]}$, and a threshold secret key share set $\{tsk_i\}_{i\in[n]}$. Each node $\mathcal{P}_i$ is assigned with $\langle tpk, \{tpk_i\}_{i\in[n]}, tsk_i \rangle$. We sometimes omit tpk, tsk_i, and tpk_i for simplicity.

- $\sigma^*_{m,i} \leftarrow \mathsf{TS.SignShare}_t(m, tsk_i)$: this algorithm generates a signature share. It takes as inputs a message m and a threshold secret key share tsk_i, and it outputs signature share $\sigma^*_{m,i}$.

- $0/1 \leftarrow \mathsf{TS.VrfyShare}_t(m, i, \sigma^*_{m,i}, tpk_i)$: this algorithm verifies a signature share. It takes as inputs a message m, an index i, a signature share $\sigma^*_{m,i}$, and a threshold public key share tpk_i. It outputs 1 iff $\sigma^*_{m,i}$ is correctly generated via $\mathsf{TS.SignShare}_t(m, tsk_i)$.

- $\sigma_m \leftarrow \mathsf{TS.Combine}_t(m, \langle i, \sigma^*_{m,i} \rangle_{i\in I})$: this algorithm generates a full signature. It takes as inputs a message m, and a set of shares $\langle i, \sigma^*_{m,i} \rangle_{i\in I}$, where $|I| > t$. The output is a full signature σ_m of message m.

- $0/1 \leftarrow \mathsf{TS.Vrfy}_t(m, \sigma_m, tpk)$: this algorithm verifies a full signature. It takes as inputs a message m, a full signature σ_m, and a threshold public key tpk. It outputs 1 iff the signature is validated by tpk.

We use Boldyreva's scheme [13] for the threshold signature, which satisfies the following properties except with negligible probability.

- *Unforgeability.* An adversary cannot forge a valid full signature of any unqueried message m.

- *Robustness.* Every honest node can generate a valid full signature as long as it receives at least $t+1$ valid signature shares generated by TS.SignShare.

Distributed pseudorandom function (DPRF) [3, 51] allows a quorum of nodes to jointly evaluate a pseudorandom function. It includes six algorithms: DPRF.Init, DPRF.Contrib, DPRF.VrfyContrib, DPRF.Combine, DPRF.Eval, and DPRF.Vrfy.

- $\langle dsk, dpk, PC_{dsk}, VC_{dpk}, \{dsk_i, dpk_i, w^i_{dsk}, \pi^i_{dpk}\}_{i\in[n]}\rangle \leftarrow \mathsf{DPRF.Init}(t, n, 1^\lambda)$: this algorithm initializes DPRF key pairs. It takes as inputs a threshold t, a committee size n, and a security parameter λ in unary form. The output is a DPRF key pair $\langle dsk, dpk\rangle$, a polynomial commitment PC_{dsk}, a vector commitment VC_{dpk}, and n tuples $\langle dsk_i, dpk_i, w^i_{dsk}, \pi^i_{dpk}\rangle_{i\in[n]}$. Here, dsk_i is $\mathcal{P}_i$'s secret key share, dpk_i is $\mathcal{P}_i$'s public key share, w^i_{dsk} is a witness to dsk_i and π^i_{dpk} is an opening proof of dpk_i. We sometimes omit dpk and dpk_i for simplicity.

- $\langle F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}\rangle \leftarrow \mathsf{DPRF.Contrib}(x, dsk_i, dpk_i, \pi^i_{dpk})$: this algorithm generates a DPRF contribution. The input is a value x, a DPRF secret key share dsk_i, a public key share dpk_i, and an opening proof π^i_{dpk} of dpk_i. The output is a DPRF contribution $F_i(x)$ and its proof $\mathsf{Proof}^i_{\mathsf{dprf}}$.

- $0/1 \leftarrow \mathsf{DPRF.VrfyContrib}(i, x, F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}})$: this algorithm verifies a DPRF contribution. It takes as inputs an index i, a value x, a contribution $F_i(x)$, and a proof $\mathsf{Proof}^i_{\mathsf{dprf}}$. It outputs 1 iff $F_i(x)$ is correctly generated via $\mathsf{DPRF.Contrib}$ with $(x, dsk_i, dpk_i, \pi^i_{dpk})$ as inputs.

- $y \leftarrow \mathsf{DPRF.Combine}(t, x, \{F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}\}_{i\in I})$: this algorithm generates a DPRF evaluation from at least $t+1$ DPRF contributions. It takes as inputs a threshold t, a value x, a contribution set $\{F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}\}_{i\in I}$, where I is the index set and $|I| > t$. The output is a DPRF evaluation $y = F(x)$.

- $y \leftarrow \mathsf{DPRF.Eval}(x, dsk)$: this algorithm directly evaluates a DPRF. It takes as inputs a value x and a DPRF secret key dsk. The output is a DPRF evaluation $y = F(x)$.

- $0/1 \leftarrow \mathsf{DPRF.Vrfy}(y, x, dpk)$: this algorithm verifies a DPRF evaluation. It takes as inputs an evaluation y, a value x, and a DPRF public key dpk. It outputs 1 iff $y = F(x)$.

We provide a specific DPRF instantiation in Algorithm 5, which is adapted from Figure 6 in the paper by Agrawal et al. [3]. A DPRF scheme satisfies the following properties except with negligible probability.

- *Consistency.* Given a fixed x, any $t+1$ valid contributions leads to the same DPRF ealuation $y = F(x)$.

- *Pseudorandomness.* A PPT adversary gains no advantage on guessing $y = F(x)$ for any unqueried x.

- *Correctness.* A PPT adversary cannot generate valid contributions that lead to an incorrect DPRF evaluation $y \neq F(x)$.

Multi-valued validated Byzantine agreement (MVBA) protocol [2, 17, 48] enables a group of nodes to submit individual proposals and reach an agreement on an output that satisfies a global predicate P_{MVBA}. The MVBA protocol satisfies the following properties except with negligible probability.

- *Termination.* If all honest nodes input proposals satisfying P_{MVBA}, then each honest node outputs a value.

- *Agreement.* If two honest nodes output in the same instance, then the output values are the same.

Algorithm 5 DPRF, adapted from Fig. 6 in [3]

Let $\mathcal{H}_1 : \{0,1\}^* \to \mathbb{G}$ and $\mathcal{H}_2 : \{0,1\}^* \to \mathbb{Z}_p$ be hash functions, where $\mathbb{G}$ is a multiplicative cyclic group with g as the generator; $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a bilinear pairing.

function $\mathsf{DPRF.Init}(t, n, 1^\lambda)$ ▷ initialize DPRF keys
  $dsk \leftarrow_\$ \mathbb{Z}_p$
  $dpk \leftarrow g^{dsk}$
  $\{PC_{dsk}, \langle dsk_i, w^i_{dsk}\rangle_{i\in[n]}\} \leftarrow \mathsf{VSS.Share}(t, n, dsk)$
  $\boldsymbol{v}_{dpk} \leftarrow \{dpk_1, \cdots, dpk_n\}$, where $dpk_i \leftarrow g^{dsk_i}$ for $i \in [n]$
  $VC_{dpk} \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}_{dpk})$
  for $i \in [n]$ **do**
    $\langle dpk_i, \pi^i_{dpk}\rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}_{dpk}, i)$
  return $\langle dsk, dpk, PC_{dsk}, VC_{dpk}, \{dsk_i, dpk_i, w^i_{dsk}, \pi^i_{dpk}\}_{i\in[n]}\rangle$

▷ verify a DPRF key share
function $\mathsf{DPRF.VrfyKey}(dsk_i, dpk_i, PC_{dsk}, VC_{dpk}, w^i_{dsk}, \pi^i_{dpk})$
  if $dpk_i = g^{dsk_i} \wedge \mathsf{VSS.Verify}(PC_{dsk}, i, dsk_i, w^i_{dsk}) = 1$ **then**
    if $\mathsf{VCom.Vrfy}(VC_{dpk}, i, dpk_i, \pi^i_{dpk}) = 1$ **then**
      return 1
  return 0

▷ generate a DPRF contribution
function $\mathsf{DPRF.Contrib}(x, dsk_i, dpk_i, \pi^i_{dpk})$
  $w \leftarrow \mathcal{H}_1(x)$
  $W_i \leftarrow w^{dsk_i}$
  $r_i \leftarrow_\$ \mathbb{Z}_p$
  $c_i \leftarrow \mathcal{H}_2(W_i, w, dpk_i, g, w^{r_i})$
  $u_i \leftarrow r_i - c_i \times dsk_i$
  denote W_i as $F_i(x)$
  $\mathsf{Proof}^i_{\mathsf{dprf}} \leftarrow \langle w, c_i, u_i, dpk_i, \pi^i_{dpk}\rangle$
  return $\langle F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}\rangle$

function $\mathsf{DPRF.VrfyContrib}(i, x, W_i, \mathsf{Proof}^i_{\mathsf{dprf}})$
  parse $\mathsf{Proof}^i_{\mathsf{dprf}}$ as $\langle w, c_i, u_i, dpk_i, \pi^i_{dpk}\rangle$
  if $\mathcal{H}_1(x) \neq w$ **then**
    return 0
  wait until receiving VC_{dpk}
  if $\mathsf{VCom.Vrfy}(VC_{dpk}, i, dpk_i, \pi^i_{dpk}) = 1$ **then**
    if $c_i = \mathcal{H}_2(W_i, w, dpk_i, g, w^{u_i} W_i^{c_i})$ **then**
      return 1
  return 0

▷ combine $t+1$ contributions
function $\mathsf{DPRF.Combine}(t, x, \{F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}\}_{i\in I})$
  if $|I| > t$ **then**
    if $\mathsf{DPRF.VrfyContrib}(i, x, F_i(x), \mathsf{Proof}^i_{\mathsf{dprf}}) = 1, \forall i \in I$ **then**
      interpolate $F(x)$ from $\{F_i(x)\}_{i\in I}$
      return $F(x)$

function $\mathsf{DPRF.Eval}(x, dsk)$ ▷ output a DPRF evaluation
  return $F(x) \leftarrow \mathcal{H}_1(x)^{dsk}$

function $\mathsf{DPRF.Vrfy}(y, x, dpk)$ ▷ verify a DPRF evaluation
  if $e(y, g) = e(\mathcal{H}_1(x), dpk)$ **then**
    return 1
  return 0

- *External-validity.* If an honest node outputs a value v, then v is external valid, i.e., $P_{\mathsf{MVBA}}(v) = 1$.

We utilize the MVBA protocol introduced by Lu et al. [48]. This protocol offers a time complexity of $O(1)$ and a message complexity of $O(n^2)$. Moreover, the communication complexity is $O(n|m| + \lambda n^2)$, where $|m|$ represents the size of input.

B The Sharing and Reconstruction Protocol

B.1 The Sharing Protocol in DPSS

In our DPSS.Handoff protocol (Section 7), we require that each old node initially holds a commitment vector $\boldsymbol{v} = \{g^{s_i}\}_{i\in[n]}$, where s_i is $\mathcal{P}_i$'s share of s. Hence, in epoch $e = 1$, we need a sharing protocol DPSS.Share that distributes valid shares and the required commitment vector among all honest nodes. We observe that the ACSS protocol introduced by Das et al. [27] meets our requirements, so we adapt their protocol in Algorithm 6.

The dealer $\mathcal{P}_d$ first invokes VSS.Share to generate n secret shares $\{s_i\}_{i\in[n]}$. Then, for each $i \in [n]$, it invokes $\mathsf{EncAndProve}(pk_i, g, s_i)$ and output $\langle v_i, c_i, \pi_i\rangle$, where $v_i = g^{s_i}$ is the commitment to s_i, c_i is the encryption of s_i using $\mathcal{P}_i$'s encryption public key pk_i, and π_i is a zero-knowledge proof that proves v_i and c_i corresponds to the same s_i. Finally, $\mathcal{P}_d$ uses a reliable broadcast instance RBC to broadcast $(\boldsymbol{v}, \boldsymbol{c}, \boldsymbol{\pi})$ to the committee $\mathcal{U}^1$, where $\boldsymbol{v} = \{v_i\}_{i\in[n]}, \boldsymbol{c} = \{c_i\}_{i\in[n]}, \boldsymbol{\pi} = \{\pi_i\}_{i\in[n]}$. When a node $\mathcal{P}_i \in \mathcal{U}^1$ outputs $(\boldsymbol{v}, \boldsymbol{c}, \boldsymbol{\pi})$ from RBC, it verifies that (1) the elements in $\boldsymbol{v}$ can be interpolated by any subset $\boldsymbol{v}' \subset \boldsymbol{v}$, $|\boldsymbol{v}'| = t + 1$, which implies $\boldsymbol{v}$ contains the commitments to a degree-t sharing polynomial; (2) the commitment-encryption pair $(\boldsymbol{v}, \boldsymbol{c})$ can be verified by $\boldsymbol{\pi}$, which ensures that all commitments are bound with the encrypted shares. If the verification pass, each node $\mathcal{P}_i$ decrypts its own share s_i from $c_i \in \boldsymbol{c}$, signs $\boldsymbol{v}$, and multicasts the signature. Finally, when $\mathcal{P}_i$ receives $t + 1$ valid and consistent signatures on $\boldsymbol{v}$, it outputs $\langle s_i, \boldsymbol{v}\rangle$. Note that the $t + 1$ signatures are also locally stored by $\mathcal{P}_i$.

The overall communication complexity of DPSS.Share is $O(\lambda n^2)$, given a RBC protocol with $O(n|m| + \lambda n^2)$ communication complexity (e.g., [25]), where $|m|$ is the input size.

B.2 The Reconstruction Protocol in DPSS

The reconstruction protocol DPSS.Recon allows a client $\mathcal{L}$ to reconstruct the secret from the new committee $\mathcal{U}'$. The client $\mathcal{L}$ has no information about the commitments to the current shares, i.e., $\boldsymbol{v}_{\mathsf{new}}$. As shown in Algorithm 7, upon invocation, every node within the current epoch (denoted by $\mathcal{P}'_i$) sends its share s'_i to the client $\mathcal{L}$, along with a vector commitment VC_{new} and an opening proof π^i_{new} of $g^{s'_i}$. The client $\mathcal{L}$ waits for $t' + 1$ valid messages with the same VC_{new}, and then it interpolates the secret s from $t' + 1$ valid shares $\{s'_i\}_{i\in I}$, where I is the index set such that $|I| = t' + 1$.

Algorithm 6 DPSS.Share, adapted from Alg. 2 in [27]

Let $\mathcal{P}_d$ be the dealer; $\mathcal{U}^1$ be the initial committee with n nodes, t of which are corrupted; $\{sk_i, pk_i\}_{i\in[n]}$ be the encryption key pairs of all nodes, where each $\mathcal{P}_i$ holds sk_i and $\{pk_i\}_{i\in[n]}$.

As a dealer $\mathcal{P}_d$:
1: **upon** receiving s as input **do**
2: generate $\{s_i\}_{i\in[n]}$ by $\mathsf{VSS.Share}(t, n, s)$
3: **for** $i \in [n]$ **do**
4: $\langle v_i, c_i, \pi_i\rangle \leftarrow \mathsf{EncAndProve}(pk_i, g, s_i)$
5: $\boldsymbol{v} \leftarrow \{v_i\}_{i\in[n]}$ ▷ $v_i = g^{s_i}$
6: $\boldsymbol{c} \leftarrow \{c_i\}_{i\in[n]}$ ▷ $c_i = \mathsf{Encrypt}(pk_i, s_i)$
7: $\boldsymbol{\pi} \leftarrow \{\pi_i\}_{i\in[n]}$ ▷ π_i proves v_i and c_i corresponds to the same s_i
8: input $(\boldsymbol{v}, \boldsymbol{c}, \boldsymbol{\pi})$ to RBC ▷ broadcast to $\mathcal{U}^1$

As a node $\mathcal{P}_i \in \mathcal{U}^1$:
9: **upon** outputting $(\boldsymbol{v}, \boldsymbol{c}, \boldsymbol{\pi})$ from RBC **do**
10: **if** all elements in $\boldsymbol{v}$ can be interpolated by any subset $\boldsymbol{v}' \subset \boldsymbol{v}$, where $|\boldsymbol{v}'| = t+1$ **then**
11: **if** the verification of $\boldsymbol{\pi}$ w.r.t. $(\boldsymbol{v}, \boldsymbol{c})$ returns true **then**
12: $s_i \leftarrow \mathsf{Decrypt}(sk_i, c_i)$ ▷ c_i is the i-th element in $\boldsymbol{c}$
13: sign $\boldsymbol{v}$ and multicast the signature
14: **wait** for $t + 1$ valid and consistent signatures on $\boldsymbol{v}$
15: **output** $\langle s_i, \boldsymbol{v}\rangle$

Algorithm 7 DPSS.Recon[ID]

Let $\boldsymbol{v}_{\mathsf{new}} = \{g^{s'_1}, ..., g^{s'_{n'}}\}$ be the commitments to the current shares

As a new node $\mathcal{P}'_i \in \mathcal{U}'$: ▷ send shares
1: **upon** receiving RECON from client $\mathcal{L}$ **do**
2: $VC_{\mathsf{new}} \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}_{\mathsf{new}})$
3: $\langle g^{s'_i}, \pi^i_{\mathsf{new}}\rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}_{\mathsf{new}}, i)$
4: send $\langle \text{RECON}, \mathsf{ID}, VC_{\mathsf{new}}, s'_i, \pi^i_{\mathsf{new}}\rangle$ to client $\mathcal{L}$

As a client $\mathcal{L}$: ▷ receive shares
5: **upon** receiving $\langle \text{RECON}, \mathsf{ID}, VC_{\mathsf{new}}, s'_i, \pi^i_{\mathsf{new}}\rangle$ from $\mathcal{P}_i$ **do**
6: **if** $\mathsf{VCom.Vrfy}(VC_{\mathsf{new}}, i, g^{s'_i}, \pi^i_{\mathsf{new}}) = 1$ **then**
7: record $\langle \mathsf{ID}, VC_{\mathsf{new}}, i, s'_i\rangle$
8: **if** recorded $t'+1$ valid tuples with same ID and VC_{new} **then**
9: interpolate s from $\{(i, s'_i)\}_{i\in I}$, I is the corresponding index set
10: **return** s

C Deferred Proofs for wpACSS

Lemma 1 (Termination of wpACSS)**.** *If the dealer $\mathcal{P}_d$ is honest, then all honest nodes output valid shares in* wpACSS.Share*, and $\mathcal{P}_d$ outputs a valid proof of sharing.*

Proof. If the dealer $\mathcal{P}_d$ is honest, then it will generate a valid value-proof tuple $\langle V_i, \Pi_i\rangle$ for each node $\mathcal{P}_i \in \mathcal{U}$ (lines 1–15, Algorithm 1, same below). Hence, every honest node $\mathcal{P}_i$ receives such a tuple containing its deserved valid share. Before $\mathcal{P}_i$ outputting this tuple, it has to receive a valid READY message which contains a valid full signature $\sigma_{r,d}$ (line 39). This condition is satisfied since the dealer $\mathcal{P}_d$ will receive at least $n - t$ ECHO messages as the responses to the SHARE messages, each containing a valid signature share (lines 29–38).

These shares are sufficient to formulate a valid signature $\sigma_{r,d}$ in READY message (lines 16–22). Therefore, if the honest dealer $\mathcal{P}_d$ invokes wpACSS.Share, all honest nodes will output valid shares in line 43.

On the other side, before the honest nodes outputting, they will each send a FINISH message to $\mathcal{P}_d$ (line 42). Each FINISH message contains a valid signature share, so $\mathcal{P}_d$ is guaranteed to formulate a full signature $\sigma_{f,d}$ (lines 23–28) and output a valid proof of sharing $\langle m_d, \sigma_{f,d} \rangle$, where m_d is generated by itself during wpACSS.Share (lines 1–17). □

Lemma 2 (Provability of wpACSS)**.** *If the dealer $\mathcal{P}_d$ outputs a valid proof of sharing from* wpACSS.Share*, then at least $t+1$ honest nodes output valid shares in* wpACSS.Share*, and these honest nodes are able to help others to recover shares via* wpACSS.BatchedRecover.

Proof. In our model, the threshold key pairs are initialized by a trusted setup. Hence, the adversary cannot generate a proof of sharing by itself. If the dealer $\mathcal{P}_d$ outputs a valid proof of sharing from wpACSS.Share, then $\mathcal{P}_d$ has received at least $n-t$ FINISH messages from distinct nodes (lines 23–28), among which $n-2t \geq t+1$ nodes are honest. These honest nodes will output value-proof tuples in line 43, each tuple contains a secret share.

We now prove the shares output by these honest nodes are valid. Without loss of generality, we denote these honest nodes as $\{\mathcal{P}_i\}_{i\in I}$, where I is the index set. The verifications in lines 32–34 ensure that each tuple $\langle V_i, \Pi_i \rangle$ is correctly formulated, i.e., the values in V_i is valid w.r.t. the commitments in Π_i, where $i \in I$. Besides, according to the verification in line 40, each node $\mathcal{P}_i$ ensures that the other nodes in $\{\mathcal{P}_i\}_{i\in I}$ receive the same m_d, which includes the commitments to the sharing and recovery polynomials (line 36). Hence, the nodes $\{\mathcal{P}_i\}_{i\in I}$ will output valid tuples that are bound with the same set of commitments. Namely, at least $t+1$ honest nodes obtain valid shares from wpACSS.Share.

Next, we prove these honest nodes are able to help others to recover shares in wpACSS.BatchedRecover. For each caller $\mathcal{P}_j$, the honest nodes $\{\mathcal{P}_i\}_{i\in I}$ will generate valid responses (lines 4–10, Algorithm 2), using the tuples $\langle V_i, \Pi_i \rangle_{i\in I}$ output from wpACSS.Share. Hence, the conditions in lines 12–14 of Algorithm 2 will be satisfied, so that $\mathcal{P}_j$ can recover its share by going through lines 15–19. □

Lemma 3 (Correctness of wpACSS)**.** *If the dealer $\mathcal{P}_d$ inputs a secret s in* wpACSS.Share *and outputs a valid proof of sharing, then there exists a fixed secret $\hat{s}$, such that any $t+1$ shares from honest nodes can reconstruct the secret $\hat{s}$. If the dealer is honest, then $s = \hat{s}$.*

Proof. By Lemma 2, we derive that if the dealer $\mathcal{P}_d$ outputs a valid proof of sharing, then every honest node can obtain output a valid share from either wpACSS.Share or wpACSS.BatchedRecover. We use $\mathcal{U}_{\mathsf{sh}}$ and $\mathcal{U}_{\mathsf{re}}$ to include the honest nodes that have outputs in wpACSS.Share and wpACSS.BatchedRecover, respectively.

We first prove that all nodes in $\mathcal{U}_{\mathsf{sh}}$ output consistent shares. An honest node $\mathcal{P}_i \in \mathcal{U}_{\mathsf{sh}}$ only outputs the value-proof tuple $\langle V_i, \Pi_i \rangle$ when it receives a valid signature $\sigma_{r,d}$ (line 39, Algorithm 1). Given there are at most t corrupted nodes, if there are two distinct message-signature pairs $\langle m_d, \sigma_{r,d} \rangle$ and $\langle m'_d, \sigma'_{r,d} \rangle$, such that $m_d \neq m'_d$ and both signatures are valid, then at least one honest node have signed to both m_d and m'_d, leading to a contradiction. Hence, there is at most one valid message-signature pair $\langle m_d, \sigma_{r,d} \rangle$, which implies that the value-proof tuples output by all honest nodes in $\mathcal{U}_{\mathsf{sh}}$ are bound with the same m_d. As m_d contains the polynomial commitment PC_s to the sharing polynomial, the shares output by the honest nodes in $\mathcal{U}_{\mathsf{sh}}$ are bound with the same PC_s.

Next, we prove the shares output by the nodes in $\mathcal{U}_{\mathsf{re}}$ are also bound with PC_s. According to the conditions in lines 12–14, Algorithm 2, an honest node $\mathcal{P}_j \in \mathcal{U}_{\mathsf{re}}$ it has to wait for $t+1$ valid HELP responses with the same commitments $(VC^d_{dpk}, PC^d_{\mathsf{mask}})$ before recovering its share. As there are at most t corrupted nodes, at least one response comes from honest nodes, which ensures the commitments are correctly computed. The vector commitment VC^d_{dpk} ensures that the DPRF evaluation in line 18 is correct. Furthermore, the polynomial commitment PC^d_{mask} guarantees that the masked share is correctly computed from the secret share and DPRF evaluation. Hence, the recovered share is consistent with the shares held by $\mathcal{U}_{\mathsf{sh}}$, the shares of both $\mathcal{U}_{\mathsf{sh}}$ and $\mathcal{U}_{\mathsf{re}}$ are bounded by PC_s.

Finally, we prove any $t+1$ shares from honest nodes lead to a fixed $\hat{s}$. We have proved that all shares output by honest nodes in $\mathcal{U}_{\mathsf{sh}}$ and $\mathcal{U}_{\mathsf{re}}$ are bound with the same polynomial commitment PC_s. Hence, there exists a sharing polynomial $\hat{f}(x)$, such that $\mathsf{PCom.Commit}(\hat{f}(x)) = PC_s$, $\hat{f}(0) = \hat{s}$, and the degree of $\hat{f}(x)$ is t. Therefore, any $t+1$ shares from honest nodes can reconstruct the secret $\hat{s}$. Moreover, if the dealer is honest, then $\hat{f}(x) = f(x)$, $s = \hat{s}$. □

Lemma 4 (Secrecy of wpACSS)**.** *If the dealer $\mathcal{P}_d$ is honest and it shares a uniformly random secret $s \in \mathbb{Z}_q$ via* wpACSS*, then a static PPT adversary learns no extra information about the secret beyond g^s.*

Proof. We prove that for a uniformly random secret s, given g^s, there exists a PPT simulator Sim_1 that can simulate the view of any static PPT adversary $\mathcal{A}$. We denote the set of corrupted nodes as $\mathcal{U}_{\mathsf{crp}}$, such that $\mathcal{U}_{\mathsf{crp}} \subset \mathcal{U}$ and $|\mathcal{U}_{\mathsf{crp}}| \leq t$.

We first construct a simulator for wpACSS.Share$[\langle \mathsf{ID}, d \rangle]$ as detailed in Figure 6. The major steps are as follows.

(1) Sim_1 runs the setups for the underlying components, including PCom, VCom, and TS. Notably, it reserves the trapdoor τ for the PCom instantiation.

(2) Sim_1 samples t uniformly random shares $\{s_i\}_{\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}}$. For ease of expression, we denote $f(x)$ as the sharing polynomial, such that $f(0) = s$ for an unknown s and $f(i) = s_i$

Simulator Sim_1 for $\mathsf{wpACSS.Share}[\langle \mathsf{ID}, d\rangle]$

1: $pp_1 \leftarrow \mathsf{PCom.Setup}(t, 1^\lambda)$
2: $pp_2 \leftarrow \mathsf{VCom.Setup}(n, 1^\lambda)$
3: $\{tpk, \{tpk_i\}_{i\in[n]}, \{tsk_i\}_{i\in[n]}\} \leftarrow \mathsf{TS.KeyGen}(t, n, 1^\lambda)$
4: Sim_1 reserves the trapdoor τ during the setup of PCom
5: Sim_1 samples t uniformly random shares $\{s_i\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$
6: Denote $f(x)$ as the sharing polynomial, such that $f(0) = s$ for an unknown s and $f(i) = s_i$ for each $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$
7: Sim_1 interpolates $PC_s = g^{f(\tau)}$ using g^s and $\{g^{s_i}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$
8: Sim_1 computes $w_s^i = g^{\frac{f(\tau)-s_i}{\tau-i}} = (\frac{PC_s}{g^{s_i}})^{(\tau-i)^{-1}}$ for each $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$
9: $PC_z \leftarrow PC_s / g^s$
10: $w_z^0 \leftarrow (PC_z)^{\tau^{-1}}$
11: $\langle dsk_i, \{\phi_k(i)\}_{k\in[\ell]}, \mathsf{Proof}_{\mathsf{rec}}^i\rangle_{i\in[n]} \leftarrow \mathsf{GenRecPoly}[\langle \mathsf{ID}, d\rangle](t, n)$
12: Sim_1 constructs the vector $\boldsymbol{v}_s$ as follows:
(1) Set $\boldsymbol{v}_s[i]$ as g^{s_i} for each $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$
(2) Interpolate $\boldsymbol{v}_s[j]$ using g^s and $\{g^{s_i}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$ for each $\mathcal{P}_j \in \mathcal{U}\backslash\mathcal{U}_{\mathsf{crp}}$
13: $VC_s \leftarrow \mathsf{VCom.Commit}(\boldsymbol{v}_s)$
14: $\langle g^{s_i}, \pi_s^i\rangle \leftarrow \mathsf{VCom.Open}(\boldsymbol{v}_s, i)$ for $i \in [n]$
15: For each corrupted node $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$:
(1) $V_i \leftarrow \langle s_i, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle$
(2) $\Pi_i \leftarrow \langle g^s, PC_s, w_s^i, PC_z, w_z^0, VC_s, \pi_s^i, \mathsf{Proof}_{\mathsf{rec}}^i\rangle$
(3) Sim_1 sends $\langle \textsc{Share}, \mathsf{ID}, V_i, \Pi_i\rangle$ to $\mathcal{P}_i$
16: For each honest node $\mathcal{P}_i \in \mathcal{U}\backslash\mathcal{U}_{\mathsf{crp}}$:
(1) $V_i \leftarrow \langle 0, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle$
(2) $\Pi_i \leftarrow \langle g^s, PC_s, \perp, PC_z, w_z^0, VC_s, \pi_s^i, \mathsf{Proof}_{\mathsf{rec}}^i\rangle$
(3) Sim_1 sends $\langle \textsc{Share}, \mathsf{ID}, V_i, \Pi_i\rangle$ to $\mathcal{P}_i$
17: Sim_1 follows lines 16–43 of Algorithm 1 to interact with $\mathcal{A}$
18: Sim_1 outputs a proof of sharing as the honest dealer $\mathcal{P}_d$, and outputs $\langle V_i, \Pi_i\rangle$ as each honest node $\mathcal{P}_i$

Figure 6: Simulator Sim_1 for $\mathsf{wpACSS.Share}[\langle \mathsf{ID}, d\rangle]$

for each $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$. Using the trapdoor τ and the $t+1$ exponents $\{g^s, \{g^{s_i}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}\}$, Sim_1 interpolates the polynomial commitment $PC_s = g^{f(\tau)}$ at position $x = \tau$. It also computes the witnesses $\{w_s^i\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$ and the commitment-witness pair $\langle PC_z, w_z^0\rangle$ of the zero polynomial.

(3) Sim_1 invokes the $\mathsf{GenRecPoly}$ function to initiate the DPRF key pairs and the recovery polynomials. In addition, Sim_1 generates the vector $\boldsymbol{v}_s$ using interpolation and computes its commitment VC_s and the opening proofs $\{\pi_s^i\}_{i\in[n]}$.

(4) With the above elements, Sim_1 can compose a valid $\textsc{Share}$ message for each corrupted node $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$, and it then interacts with the corrupted nodes following lines 16–43 of Algorithm 1 on behalf of the simulated dealer $\mathcal{P}_d$. Notably, Sim_1 uses the threshold signing keys of honest nodes to generate the $\textsc{Ready}$ messages sent to the corrupted nodes. In the end, Sim_1 outputs a proof of sharing as the honest dealer $\mathcal{P}_d$, and outputs $\langle V_i, \Pi_i\rangle$ as each honest node $\mathcal{P}_i$.

Next, we prove that the view of $\mathcal{A}$ in its interaction with Sim_1 is computationally indistinguishable from its view in the real-world protocol. In the adversary's view, it receives t $\textsc{Share}$ and $\textsc{Ready}$ messages from $\mathcal{P}_d$. Each $\textsc{Share}$ message contains a value-proof tuple $\langle V_i, \Pi_i\rangle$, where $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$. For ease of comparison, we denote the simulated tuple as $\langle \hat{V}_i, \hat{\Pi}_i\rangle$. The adversary's task is to distinguish the following two tuples.

$$V_i = \langle s_i, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle,$$
$$\Pi_i = \langle g^s, PC_s, w_s^i, PC_z, w_z^0, VC_s, \pi_s^i, \mathsf{Proof}_{\mathsf{rec}}^i\rangle$$

and

$$\hat{V}_i = \langle \hat{s}_i, dsk_i, \{\phi_k(i)\}_{k\in[\ell]}\rangle,$$
$$\hat{\Pi}_i = \langle g^s, PC_{\hat{s}}, w_{\hat{s}}^i, PC_{\hat{z}}, w_{\hat{z}}^0, VC_{\hat{s}}, \pi_{\hat{s}}^i, \mathsf{Proof}_{\mathsf{rec}}^i\rangle$$

Note that the elements $\langle dsk_i, \{\phi_k(i)\}_{k\in[\ell]}, \mathsf{Proof}_{\mathsf{rec}}^i\rangle$ are honestly generated by the $\mathsf{GenRecPoly}$ function in both worlds, so they are computationally indistinguishable. Since both $s_i \in V_i$ and $\hat{s}_i \in \hat{V}_i$ are uniformly random, and the simulated elements $\langle PC_{\hat{s}}, w_{\hat{s}}^i, PC_{\hat{z}}, w_{\hat{z}}^0, VC_{\hat{s}}, \pi_{\hat{s}}^i\rangle \in \hat{\Pi}_i$ are ensured to pass all validations in lines 32–34, Algorithm 1, the adversary cannot distinguish these elements in the two tuples. Therefore, the two tuples are computationally indistinguishable. Then, in each $\textsc{Ready}$ message, there is a combination of commitments and an honestly generated signature, where the commitments are extracted from Π_i. Hence, the indistinguishability of the value-proof tuples also implies that the $\textsc{Ready}$ messages are also indistinguishable by $\mathcal{A}$.

In the $\mathsf{wpACSS.BatchedRecover}$ sub-protocol, the adversary $\mathcal{A}$ may call for help on behalf of corrupted nodes. As claimed in VSSR [8], the responses from honest nodes reveal no information about the secret. Hence, we omit the simulation and the indistinguishability discussion for $\mathsf{wpACSS.BatchedRecover}$ for simplicity.

Altogether, the view of the adversary $\mathcal{A}$ in the interaction with the simulator is computationally indistinguishable from the view in real-world execution. Hence, we conclude that $\mathcal{A}$ learns no extra information about the uniformly random secret s beyond g^s. □

Combining Lemma 1–4, we get Theorem 5 as below.

Theorem 5. *The tuple of protocols described in Algorithm 1 and Algorithm 2 satisfies the termination, provability, correctness, and secrecy properties of wpACSS except with negligible probability.*

D Deferred Proofs for DPSS.Handoff

Lemma 6 (Termination of handoff)**.** *If all honest nodes in committees $\mathcal{U}$ and $\mathcal{U}'$ invoke* DPSS.Handoff*, then all honest old nodes halt, and all honest new nodes have outputs.*

Proof. We first prove that all honest old nodes halt in the handoff protocol. Following our adversary model, there are three types of old nodes: (1) corrupted nodes, (2) honest in epoch $e-1$ but corrupted in epoch e, and (3) the others. The term "honest old node" refers to the third type. In current epoch e, there are at least $t+1$ honest old nodes, and these nodes have old shares and a common commitment vector $\boldsymbol{v}_{\mathsf{old}}$. Hence, these honest nodes can compose the $\textsc{Com}$ messages and invoke $\mathsf{wpACSS.Share}$ with their old shares as inputs (lines 5–6, Algorithm 3). By the termination of wpACSS (Lemma 1), every honest node $\mathcal{P}_i$ that invokes $\mathsf{wpACSS.Share}$ is guaranteed

to receive a proof of sharing as output, which is used to formulate the PROOF message. Hence, all honest old nodes will multicast the necessary messages and halt.

Then, we prove that all honest new nodes have outputs in the handoff protocol. Since at least $t+1$ honest old nodes have the same commitment vector $\boldsymbol{v}_{\mathsf{old}}$, each honest new node $\mathcal{P}_i'$ is guaranteed to receive $t+1$ valid COM messages with the same VC_{old} and interpolate the commitments in line 10. Besides, the honest new nodes will receive at least $t+1$ valid PROOF messages, so all honest new nodes input valid proposals to MVBA (lines 11–16). By the termination of MVBA, each honest new node obtains the same proposal $\overline{W}$ and index set I (lines 17–18). For each $j \in I$, there exists a valid proof of sharing $\langle m_j, \bar{\sigma}_j \rangle \in \overline{W}$. Hence, if $\mathcal{P}_i'$ invokes wpACSS.BatchedRecover (line 23), it is guaranteed to recover the missing shares by Lemma 2. Then, $\mathcal{P}_i'$ is ensured to obtain all sub-shares $\{s_{j,i}\}_{j\in I}$ and interpolate its new share s_i' (line 24). Afterward, each $\mathcal{P}_i'$ calls GetNewCom (Algorithm 4) to generate the new commitment vector $\boldsymbol{v}_{\mathsf{new}}$.

In GetNewCom, if all new nodes multicast valid NEWCOM messages, then each $\mathcal{P}_i'$ interpolates the correct $g^{s_0'}$ and returns $\boldsymbol{v}_{\mathsf{new}}$ (lines 1–4, Algorithm 4), leading to the output of DPSS.Handoff. If any ERROR message appears, each new node waits for $t'+1$ sets of $\{g^{s_{k,j}}\}_{k\in I}$ for distinct j (lines 12–14, Algorithm 4). When an honest node $\mathcal{P}_i'$ receives a valid AUX message from $\mathcal{P}_j'$, there might be two cases: (1) $I_j = I$ and (2) $I_j \neq I$. In the first case, the j-th commitment $g^{s_j'}$ can be directly interpolated from the information in the AUX message (lines 19–20, Algorithm 4). As for the second case, since there exists a valid proof of sharing for each $k \in I \backslash I_j$, at least $t'+1$ honest new nodes have output valid shares, and they will contribute $g^{s_{k,*}}$ via the AUX messages. Upon collecting $t'+1$ such valid AUX messages that contain $g^{s_{k,*}}$, $\mathcal{P}_i'$ interpolates the missing $g^{s_{k,j}}$ and enter lines 19–20 (Algorithm 4) to compute $g^{s_j'}$. In both cases, $\mathcal{P}_i'$ obtains the j-th new commitment. Hence, $\mathcal{P}_i'$ can collect at least $t'+1$ new commitments and compute the new vector $\boldsymbol{v}_{\mathsf{new}}$. □

Lemma 7 (Correctness of handoff)**.** *The secret value s stays invariant across the executions of* DPSS.Handoff.

Proof. The adversary has the ability to eliminate the commitment vector $\boldsymbol{v}_{\mathsf{old}}$ or forge a fake vector in the corrupted nodes. However, considering that the adversary can corrupt at most t nodes, they cannot generate $t+1$ valid signatures. Hence, at the beginning of each DPSS.Handoff, every honest old node checks whether it has $t+1$ valid signatures on the commitment vector $\boldsymbol{v}_{\mathsf{old}}$, ensuring the consistency and correctness of $\boldsymbol{v}_{\mathsf{old}}$. Then, following the procedures of DPSS.Handoff, the new nodes obtain a common index set I (line 18, Algorithm 3), where $|I| = t+1$. For each $j \in I$, the old node $\mathcal{P}_j$ reshares its share s_j using a degree-t' sharing polynomial $f_j(x)$, such that $f_j(0) = s_j$. The commitment vector $\boldsymbol{v}_{\mathsf{old}}$ ensures that the input to each wpACSS.Share instance is the correct old share. By the external validity of MVBA, the proofs of sharing from $\{\mathcal{P}_j\}_{j\in I}$ are all valid. Hence, by the provability of wpACSS (Lemma 2), each new node $\mathcal{P}_i'$ outputs valid shares $\{s_{j,i}\}_{j\in I}$, where $s_{j,i} = f_j(i)$. The $t+1$ polynomials $\{f_j(x)\}_{j\in I}$ form a degree-$\langle t', t\rangle$ bivariate polynomial $B(x,y)$, such that $B(x,j) = f_j(x)$. Each new node $\mathcal{P}_i'$ interpolates its new share $s_i' = B(i,0)$ from $\{s_{j,i}\}_{j\in I} = \{B(i,j)\}_{j\in I}$. Besides, the old share held by $\mathcal{P}_i$ can be represented by $s_i = B(0,i)$. Hence, the secret value $s = B(0,0)$ can be reconstructed from $t+1$ old shares or $t'+1$ new shares, so the secret value s remains invariant. □

Simulator Sim_2 for DPSS.Handoff[ID]

1: Sim_2 obtains $\{s_i, \boldsymbol{v}_{\mathsf{old}}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$ of the prior epoch from the simulated execution of DPSS.Handoff, where $\boldsymbol{v}_{\mathsf{old}} = \{g^{s_1}, g^{s_2}, ..., g^{s_n}\}$
2: Sim_2 initialize the setups for PCom, VCom, TS, and PKI, and it reserves the trapdoor τ of PCom
3: Sim_2 simulates each honest old node $\mathcal{P}_i$ following lines 1–7, Algorithm 3. Specially, Sim_2 runs Sim_1 for each wpACSS.Share[$\langle$ID, $i\rangle$] instance, with $g^{s_i} \in \boldsymbol{v}_{\mathsf{old}}$ as input
4: Sim_2 simulates honest new nodes following lines 8–17, Algorithm 3, and obtains the common index set I
5: Sim_2 runs the wpACSS.BatchedRecover instances (possibly invoked by $\mathcal{A}$) following lines 18–23, Algorithm 3
6: Sim_2 extracts the new shares $\{s_i'\}_{\mathcal{P}_i'\in\mathcal{U}_{\mathsf{crp}}'}$ of corrupted nodes as follows:
(1) For each honest dealer $\mathcal{P}_j$ satisfying $j \in I$, Sim_2 already knows the simulated subshare $s_{j,i}$ held by each corrupted new node $\mathcal{P}_i'$
(2) For each corrupted dealer $\mathcal{P}_j$ satisfying $j \in I$, since there exists a proof of share, at least $t'+1$ simulated honest new nodes have received valid subshares. Sim_2 uses these subshares to reconstruct the sharing polynomial $f_j(x)$ and then evaluates $s_{j,i} = f_j(i)$ for $\mathcal{P}_i' \in \mathcal{U}_{\mathsf{crp}}'$
(3) The new share s_i' for $\mathcal{P}_i'$ is interpolated from $\{(j, s_{j,i})\}_{j\in I}$
7: Sim_2 constructs the new commitment vector $\boldsymbol{v}_{\mathsf{new}}$ as follows:
(1) Set $\boldsymbol{v}_{\mathsf{new}}[i]$ as $g^{s_i'}$ for each $\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}$
(2) Interpolate $\boldsymbol{v}_{\mathsf{new}}[j]$ using g^s and $\{g^{s_i'}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$ for each $\mathcal{P}_j \in \mathcal{U}\backslash\mathcal{U}_{\mathsf{crp}}$
8: Sim_2 runs lines 25–28, Algorithm 3, using $\boldsymbol{v}_{\mathsf{new}}$

Figure 7: Simulator Sim_2 for DPSS.Handoff[ID]

Lemma 8 (Secrecy of handoff)**.** *Given a uniformly random secret s, a static PPT adversary does not gain any additional knowledge about the secret from* DPSS.Handoff *beyond g^s.*

Proof. We prove that for a uniformly random secret s, given g^s, there exists a PPT simulator Sim that can simulate the view of any static PPT adversary $\mathcal{A}$. We denote the sets of corrupted nodes in the old and new committees as $\mathcal{U}_{\mathsf{crp}}$ and $\mathcal{U}_{\mathsf{crp}}'$, respectively, such that $\mathcal{U}_{\mathsf{crp}} \subset \mathcal{U}$, $\mathcal{U}_{\mathsf{crp}}' \subset \mathcal{U}'$, $|\mathcal{U}_{\mathsf{crp}}| \leq t$, and $|\mathcal{U}_{\mathsf{crp}}'| \leq t'$.

We design the simulator Sim_2 for our DPSS.Handoff as detailed in Figure 7. The specific steps are as follows.

(1) Sim_2 first obtains $\{s_i, \boldsymbol{v}_{\mathsf{old}}\}_{\mathcal{P}_i\in\mathcal{U}_{\mathsf{crp}}}$ of the prior epoch from the simulated execution of DPSS.Handoff. Then, Sim_2 initializes the underlying components, including PCom, VCom, TS, and the PKI for signing keys. Besides, Sim_2 reserves the trapdoor τ for PCom.

(2) Sim_2 simulates each honest old node $\mathcal{P}_i$, following lines 1–7, Algorithm 3. Specially, for each invocation of wpACSS.Share[$\langle$ID, $i\rangle$], Sim_2 runs Sim_1 with $g^{s_i} \in \boldsymbol{v}_{\mathsf{old}}$ as input. Sim_2 also simulates the honest new nodes following lines 8–23, running the MVBA[ID] instance and any

wpACSS.BatchedRecover instances invoked by either honest nodes or the adversary. The common index set obtained in line 17 is denoted as I.

(3) Sim_2 extracts the new shares $\{s'_i\}_{\mathcal{P}'_i \in \mathcal{U}'_{\mathsf{crp}}}$ of corrupted new nodes as follows. For each honest dealer $\mathcal{P}_j$ identified by I, Sim_2 already knows the simulated subshare $s_{j,i}$ held by each corrupted node $\mathcal{P}'_i$. For each corrupted dealer $\mathcal{P}_j$ identified by I, due to the external validity of MVBA, there exists a valid proof of sharing for $\mathcal{P}_j$, so at least $t'+1$ honest new nodes have received a valid subshare from $\mathcal{P}_j$. Sim_2 then uses these subshares to reconstruct the sharing polynomial $f_j(x)$ and then evaluates $s_{j,i} = f_j(i)$ for each $\mathcal{P}'_i \in \mathcal{U}'_{\mathsf{crp}}$. Once Sim_2 recovers all subshares sent by nodes $\{\mathcal{P}_j\}_{j\in I}$, it interpolates the new share s'_i for each $\mathcal{P}'_i \in \mathcal{U}'_{\mathsf{crp}}$ from $\{(j, s_{j,i})\}_{j\in I}$.

(4) Sim_2 constructs the new commitment vector $\boldsymbol{v}_{\mathsf{new}}$ using g^s and $\{g^{s'_i}\}_{\mathcal{P}_i \in \mathcal{U}_{\mathsf{crp}}}$, where s'_i is obtained in the prior step. Finally, Sim_2 runs lines 25–28, Algorithm 3, using $\boldsymbol{v}_{\mathsf{new}}$, which includes the GetNewCom[ID] instance and the interactions to obtain $t'+1$ signatures on $\boldsymbol{v}_{\mathsf{new}}$.

Next, we prove that the view of $\mathcal{A}$ in its interaction with Sim_2 is computationally indistinguishable from its view in the real-world protocol.

During the execution of DPSS.Handoff, $\mathcal{A}$ receives $n-t$ COM messages (line 5) , $n-t$ PROOF messages (line 7), and it interacts with the honest nodes in n wpACSS.Share instances (line 6), one MVBA instance (line 16), several optional wpACSS.BatchedRecover instances (line 23), and one GetNewCom instance (line 25). Moreover, $\mathcal{A}$ interacts with honest nodes through lines 26–28.

Firstly, since the old shares in both worlds are uniformly random, the simulated vector $\boldsymbol{v}_{\mathsf{old}}$ has the same distribution as that in the real world. Therefore, the COM messages, which are honestly generated using the vector $\boldsymbol{v}_{\mathsf{old}}$ in both worlds, are indistinguishable by the adversary. Next, by the secrecy of wpACSS (Lemma 4), the wpACSS.Share and wpACSS.BatchedRecover instances are indistinguishable. As the PROOF messages each contains an output of wpACSS.Share instance, they are also indistinguishable by $\mathcal{A}$. Besides, as the MVBA instance is honestly executed, the views of $\mathcal{A}$ within MVBA in the two worlds are identical. Finally, the messages in the GetNewCom instance are derived from the simulated elements in the wpACSS.Share instances, so the adversary cannot distinguish whether the GetNewCom instance is executed in the real world or the ideal world.

Altogether, the view of $\mathcal{A}$ in the interaction with the simulator is computationally indistinguishable from the view in real-world execution. Hence, $\mathcal{A}$ does not gain any extra knowledge about the secret from DPSS.Handoff beyond g^s. □

Combining Lemma 6–8, we obtain Theorem 9 as below.

Theorem 9. *The protocol described in Algorithm 3 satisfies the termination, correctness, and secrecy properties of the* DPSS.Handoff *protocol except with negligible probability.*

DPSS (cf. Section 7)	**bDPSS (cf. Section 8.1)**
1: Each $\mathcal{P}_i$ reshares its old share s_i to the new committee. $\mathcal{P}_i$ multicasts the proofs of sharing to the new committee.	1: Each $\mathcal{P}_i$ shares $B/(n-2t)$ random values to both old and new committees, using different polynomials. $\mathcal{P}_i$ multicasts the proofs of sharing to the old committee.
2: The new committee runs MVBA to identify $t+1$ old nodes who correctly reshare their old shares.	2: The old committee runs MVBA to identify $n-t$ old nodes who correctly share all random values. The old committee disseminates the index set I to the new committee.
	3: Each $\mathcal{P}_i$ (resp., $\mathcal{P}'_i$) extracts B shares, denoted as $\{r_{k,i}\}_{k\in[B]}$ (resp., $\{r'_{k,i}\}_{k\in[B]}$) based on the index set I.
	4: The old committee reconstructs $\{s_k + r_k\}_{k\in[B]}$ and sends them to the new committee.
3: Each $\mathcal{P}'_i$ computes its new share s'_i according to the MVBA output.	5: Each $\mathcal{P}'_i$ waits for $t+1$ consistent reconstructed values and computes $\{s'_{k,i} = s_k + r_k - r'_{k,i}\}_{k\in[B]}$ as its new shares.
4: The new nodes interact to generate the commitment vector $\boldsymbol{v}_{\mathsf{new}}$, which contains commitments to all new shares.	

Figure 8: Comparison of DPSS and bDPSS (B is batch size).

E Details of our Batched DPSS

Algorithm 8 shows details of bDPSS.Handoff described in Section 8.1, where B is the batch size and M is a hyperinvertible matrix (referred to as H matrix). The primary differences between bDPSS and DPSS is highlighted in Figure 8. The detailed procedure is as follows:

- Each old node $\mathcal{P}_i$ locally samples $B/(n-2t)$ random values, denoted as $\{r^i_{\mathsf{local},k}\}_{k\in[B/(n-2t)]}$. For each $k \in [B/(n-2t)]$, $\mathcal{P}_i$ shares the random value $r^i_{\mathsf{local},k}$ to $\mathcal{U}$ (resp., $\mathcal{U}'$) with a threshold t (resp., t'). At the end of sharing, $\mathcal{P}_i$ outputs a set of proofs of sharing $\langle m_{k,i}, m'_{k,i}, \sigma_{\mathsf{f},k,i}, \sigma'_{\mathsf{f},k,i}\rangle_{k\in[B/(n-2t)]}$, where $m_{k,i}$ and $m'_{k,i}$ have a common prefix $\mathsf{ID}||i||g^{r^i_{\mathsf{local},k}}$. This proof is multicasted to the old committee.

- The old committee collects $n-t$ proofs of sharing and then runs MVBA to identify $n-t$ old nodes who correctly shares all $B/(n-2t)$ random values. The predicate of MVBA requires that all signatures in the proofs of sharing are valid, and every pair of $\langle m_{k,d}, m'_{k,d}\rangle$ has the same prefix, ensuring that the dealer (old node) $\mathcal{P}_d$ shares the same $r^d_{\mathsf{local},k}$ to the old and new committees. Then, the old committee aggregates the indexes of the $n-t$ selected nodes into I, and disseminates it to the new committee. Note that directly multicasting I would incur $O(\lambda n^3)$ communication complexity, so we use error-correcting codes to keep the optimistic communication

Algorithm 8 bDPSS.Handoff[ID] between $\mathcal{U}$ and $\mathcal{U}'$, with B as the batch size and M as a hyperinvertible matrix

Let $\mathcal{U}$ be the old committee with n nodes, t of which are corrupted;
Let $\mathcal{U}'$ be the new committee with n' nodes, t' of which are corrupted;
Let the predicate of MVBA be: $P_{\mathsf{MVBA}}(W) \equiv (W$ contains $n-t$ sets of $\{m_{k,i}, m'_{k,i}, \sigma_{k,i}, \sigma'_{k,i}\}_{k\in[B/(n-2t)]}$ for distinct indexes i, such that every pair of $m_{k,i}$ and $m'_{k,i}$ have the same prefix $\mathsf{ID}||i||g^{r^i_{\mathsf{local},k}}$, and $\langle \sigma_{k,i}, \sigma'_{k,i}\rangle$ are valid w.r.t. $\langle m_{k,i}, m'_{k,i}\rangle$)

As an old node $\mathcal{P}_i \in \mathcal{U}$:
Init: $W_i \leftarrow \emptyset$; $I_i \leftarrow \emptyset$; $S^i_{\mathsf{help}} \leftarrow \emptyset$; start $\mathsf{wpACSS.Share}[\langle \mathsf{ID}, d, k, 1\rangle]$ for all $d \in [n]$ and $k \in [B/(n-2t)]$ to receive shares

▷ share $B/(n-2t)$ random values

1: **upon** receiving batch size B as input **do**
2: **for** $k \in [B/(n-2t)]$ **do**
3: randomly sample $r^i_{\mathsf{local},k}$
4: $\langle m_{k,i}, \sigma_{k,i}\rangle \leftarrow \mathsf{wpACSS.Share}[\langle \mathsf{ID}, i, k, 1\rangle](\mathcal{U}, t, n, r^i_{\mathsf{local},k})$
5: $\langle m'_{k,i}, \sigma'_{k,i}\rangle \leftarrow \mathsf{wpACSS.Share}[\langle \mathsf{ID}, i, k, 2\rangle](\mathcal{U}', t', n', r^i_{\mathsf{local},k})$
6: multicast $\langle \textsc{Proof}, \mathsf{ID}, \{m_{k,i}, m'_{k,i}, \sigma_{k,i}, \sigma'_{k,i}\}_{k\in[B/(n-2t)]}\rangle$ to $\mathcal{U}$

7: **upon** receiving $\langle \textsc{Proof}, \mathsf{ID}, \{m_{k,j}, m'_{k,j}, \sigma_{k,j}, \sigma'_{k,j}\}_{k\in[B/(n-2t)]}\rangle$ from $\mathcal{P}_j$ **do**
8: **if** for each $k \in [B/(n-2t)]$, $m_{k,j}$ and $m'_{k,j}$ have the same prefix $\mathsf{ID}||j||g^{r^j_{\mathsf{local},k}} \wedge \langle \sigma_{k,j}, \sigma'_{k,j}\rangle$ are valid w.r.t. $\langle m_{k,j}, m'_{k,j}\rangle$ **then**
9: $I_i \leftarrow I_i \cup \{j\}$
10: $W_i \leftarrow W_i \cup \{m_{k,j}, m'_{k,j}, \sigma_{k,j}, \sigma'_{k,j}\}_{k\in[B/(n-2t)]}$
11: **if** $|I_i| = n-t$ **then**
12: input W_i into $\mathsf{MVBA}[\mathsf{ID}]$

13: **upon** receiving $\overline{W}$ from $\mathsf{MVBA}[\mathsf{ID}]$ **do**
14: let I be the index set corresponds to $\overline{W}$, such that $|I| = n-t$
15: **if** not yet receive any of $\{r^j_{\mathsf{local},k,i}\}_{k\in[B/(n-2t)],j\in I}$ **then**
16: recover the missing shares by $\mathsf{wpACSS.BatchedRecover}$
17: **for** $k \in [B/(n-2t)]$ **do**
18: $[r_{k,1,i}, r_{k,2,i}, \cdots, r_{k,n-2t,i}] \leftarrow M \cdot [\{r^j_{\mathsf{local},k,i}\}_{j\in I}]$
19: reorder $[r_{k,1,i}, r_{k,2,i}, \cdots, r_{k,n-2t,i}]_{k\in[B/(n-2t)]}$ as $\{r_{k,i}\}_{k\in[B]}$
20: **for** $k \in [B]$ **do**
21: $s_{\mathsf{mask},k,i} \leftarrow s_{k,i} + r_{k,i}$ ▷ used in batch reconstruction
22: batch-reconstruct $\{s_k + r_k\}_{k\in[B]}$ ▷ cf. ReconsPubl in [10]
23: multicast $\langle \textsc{Mask}, \{s_k + r_k\}_{k\in[B]}\rangle$ to $\mathcal{U}'$
24: $\{m_1, \cdots, m_n\} \leftarrow \mathsf{RSEnc}(t, n, I)$ ▷ encode I using RS code
25: multicast $\langle \textsc{Ecc}, \mathsf{ID}, m_i\rangle$ to $\mathcal{U}'$
26: delete all private information and halt ▷ halt

As a new node $\mathcal{P}'_i \in \mathcal{U}'$:
Init: start $\mathsf{wpACSS.Share}[\langle \mathsf{ID}, d, k, 2\rangle]$ for all $d \in [n]$ and $k \in [B/(n-2t)]$ to receive shares

27: **for** $0 \le r \le t$ **do**
28: **upon** receiving $2t+1+r$ $\textsc{Ecc}$ messages from distinct $\mathcal{P}_i$ **do**
29: run Online-Error-Correcting (OEC) algorithm until obtain I
30: **upon** receiving I **do**
31: follow line 15–19 to compute $\{r'_{k,i}\}_{k\in[B]}$
32: **wait** for $t+1$ $\textsc{Mask}$ messages with the same $\{s_k + r_k\}_{k\in[B]}$
33: **for** $k \in [B]$ **do**
34: $s'_{k,i} \leftarrow s_k + r_k - r'_{k,i}$
35: **output** $\{s'_{k,i}\}_{k\in[B]}$

complexity within $O(\lambda n^2)$.

- For each $k \in [B/(n-2t)]$, the old node $\mathcal{P}_i$ multiplies the H matrix with the $n-t$ selected shares. The multiplication for a specific k yields $n-2t$ random shares. Hence, each $\mathcal{P}_i$ obtains B random shares $\{r_{k,i}\}_{k\in[B]}$, where $r_{k,i}$ is the share of a uniformly random value r_k. Each new node $\mathcal{P}'_i$ receives the index set I from the old committee and then extracts B random shares $\{r'_{k,i}\}_{k\in[B]}$ in the same way.
- Each old node $\mathcal{P}_i$ masks their old shares by computing $\{s_{k,i}+r_{k,i}\}_{k\in[B]}$. Then, $\mathcal{P}_i$ reconstructs $\{s_k+r_k\}_{k\in[B]}$ using the batched reconstruction algorithm in [10]. The reconstructed values are sent to the new committee.
- Each new node $\mathcal{P}'_i$ waits for $t+1$ messages with the same reconstructed values and computes the new share $s'_{k,i} = s_k + r_k - r'_{k,i}$ for every $k \in [B]$. The new nodes are no longer required to generate the commitment vector $\boldsymbol{\nu}_{\mathsf{new}}$ because the inputs for the provable sharing instances are random values rather than the old shares.